Topic : Predictive Analytics of Heart Disease Mortality A Dual Approach Using Regression and Classification Machine Learning Models

1. IMPORT LIBRARIES

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, GridSearchCV, Randomizec
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.feature_selection import SelectKBest, f_regression

# Regression Models
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from xgboost import XGBRegressor

# Classification Models
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifie
from sklearn.neighbors import KNeighborsClassifier
from sklearn.neural_network import MLPClassifier

# Metrics
from sklearn.metrics import *
```

In [2]:
```
import os
os.chdir("C:/Users/noah/Downloads")
```

2. LOAD & MERGE DATA

In [3]:
```
df1 = pd.read_csv("2015.csv")
df2 = pd.read_csv("2016.csv")
df3 = pd.read_csv("2017.csv")

df = pd.concat([df1, df2, df3], ignore_index=True)
```

3. DATA INSPECTION

In [4]:
```
print("Shape:", df.shape)
print("\nColumns:\n", df.columns)
print("\nInfo:\n")
print(df.info())

print("\nFirst Rows:\n", df.head())
print("\nStatistical Summary:\n", df.describe())
```

```
Shape: (177264, 22)

Columns:
 Index(['Year', 'LocationAbbr', 'LocationDesc', 'GeographicLevel', 'DataSourc
e',
       'Class', 'Topic', 'Data_Value', 'Data_Value_Unit', 'Data_Value_Type',
       'Data_Value_Footnote_Symbol', 'Data_Value_Footnote',
       'StratificationCategory1', 'Stratification1', 'StratificationCategory2',
       'Stratification2', 'TopicID', 'LocationID', 'Location 1', 'Y_lat',
       'X_lon', 'Georeference Column'],
      dtype='object')

Info:

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 177264 entries, 0 to 177263
Data columns (total 22 columns):
 #   Column                      Non-Null Count   Dtype
---  ------                      --------------   -----
 0   Year                        177264 non-null  int64
 1   LocationAbbr                177264 non-null  object
 2   LocationDesc                177264 non-null  object
 3   GeographicLevel             177264 non-null  object
 4   DataSource                  177264 non-null  object
 5   Class                       177264 non-null  object
 6   Topic                       177264 non-null  object
 7   Data_Value                  96679 non-null   float64
 8   Data_Value_Unit             177264 non-null  object
 9   Data_Value_Type             177264 non-null  object
 10  Data_Value_Footnote_Symbol  80585 non-null   object
 11  Data_Value_Footnote         80585 non-null   object
 12  StratificationCategory1     177264 non-null  object
 13  Stratification1             177264 non-null  object
 14  StratificationCategory2     177264 non-null  object
 15  Stratification2             177264 non-null  object
 16  TopicID                     177264 non-null  object
 17  LocationID                  177264 non-null  int64
 18  Location 1                  59058 non-null   object
 19  Y_lat                       118152 non-null  float64
 20  X_lon                       118152 non-null  float64
 21  Georeference Column         59076 non-null   object
dtypes: float64(3), int64(2), object(17)
memory usage: 29.8+ MB
None

First Rows:
    Year LocationAbbr   LocationDesc GeographicLevel DataSource  \
0  2015           CO       Colorado           State       NVSS
1  2015           ME          Maine           State       NVSS
2  2015           NV    Carson City          County       NVSS
3  2015           OH           Ohio           State       NVSS
4  2015           US  United States          Nation       NVSS

                      Class                     Topic  Data_Value  \
```

```
0  Cardiovascular Diseases  Heart Disease Mortality       326.0
1  Cardiovascular Diseases  Heart Disease Mortality       247.0
2  Cardiovascular Diseases  Heart Disease Mortality       338.1
3  Cardiovascular Diseases  Heart Disease Mortality       417.7
4  Cardiovascular Diseases  Heart Disease Mortality       409.0

          Data_Value_Unit                                     Data_Value_Type  \
0  per 100,000 population                  Age-adjusted, 3-year Average Rate
1  per 100,000 population                  Age-adjusted, 3-year Average Rate
2  per 100,000 population  Age-adjusted, Spatially Smoothed, 3-year Avera...
3  per 100,000 population                  Age-adjusted, 3-year Average Rate
4  per 100,000 population                  Age-adjusted, 3-year Average Rate

   ... StratificationCategory1 Stratification1 StratificationCategory2  \
0  ...                     Sex            Male          Race/Ethnicity
1  ...                     Sex            Male          Race/Ethnicity
2  ...                     Sex         Overall          Race/Ethnicity
3  ...                     Sex         Overall          Race/Ethnicity
4  ...                     Sex         Overall          Race/Ethnicity

  Stratification2 TopicID LocationID                Location 1  Y_lat X_lon  \
0           White      T2          8        (38.997, -105.549)    NaN   NaN
1           Black      T2         23       (45.3889, -69.2349)    NaN   NaN
2           Black      T2      32510  (39.149606, -119.741553)    NaN   NaN
3           Black      T2         39       (40.2882, -82.7866)    NaN   NaN
4           Black      T2          0                       NaN    NaN   NaN

   Georeference Column
0                  NaN
1                  NaN
2                  NaN
3                  NaN
4                  NaN

[5 rows x 22 columns]

Statistical Summary:
                 Year    Data_Value     LocationID          Y_lat  \
count  177264.000000  96679.000000  177264.000000  118152.000000
mean     2016.000102    344.653510   30930.366673      37.901745
std         0.816478    142.754854   16734.631170       6.326368
min      2015.000000      0.000000       0.000000     -14.301754
25%      2015.000000    246.700000   18135.000000      34.326243
50%      2016.000000    333.600000   29206.000000      38.220930
75%      2017.000000    428.100000   46089.000000      41.695739
max      2017.000000   5161.700000   78030.000000      69.309529

               X_lon
count  118152.000000
mean      -91.406452
std        15.942235
min      -170.719474
25%       -98.129615
50%       -89.931055
```

```
75%       -82.890640
max       145.751259
```

4. CHECK MISSING VALUES

In [5]:
```
print("\nMissing Values:\n", df.isnull().sum())
```

```
Missing Values:
 Year                              0
LocationAbbr                      0
LocationDesc                      0
GeographicLevel                   0
DataSource                        0
Class                             0
Topic                             0
Data_Value                    80585
Data_Value_Unit                   0
Data_Value_Type                   0
Data_Value_Footnote_Symbol    96679
Data_Value_Footnote           96679
StratificationCategory1           0
Stratification1                   0
StratificationCategory2           0
Stratification2                   0
TopicID                           0
LocationID                        0
Location 1                   118206
Y_lat                         59112
X_lon                         59112
Georeference Column          118188
dtype: int64
```

5. CLEAN COLUMN NAMES

In [6]:
```
df.columns = df.columns.str.lower().str.replace(" ", "_")
```

6. FILTER RELEVANT DATA

In [7]:
```
df = df[df['topic'].str.contains("heart", case=False, na=False)]
```

7. DROP UNNECESSARY COLUMN

In [8]:
```
drop_cols = [
    "DataSource", "TopicID", "LocationID", "Y_lat", "X_lon",
       "Georeference",  "Data_Value_Footnote","LocationAbbr","Data_Value_Footr

df.drop(columns=drop_cols, inplace=True, errors='ignore')
```

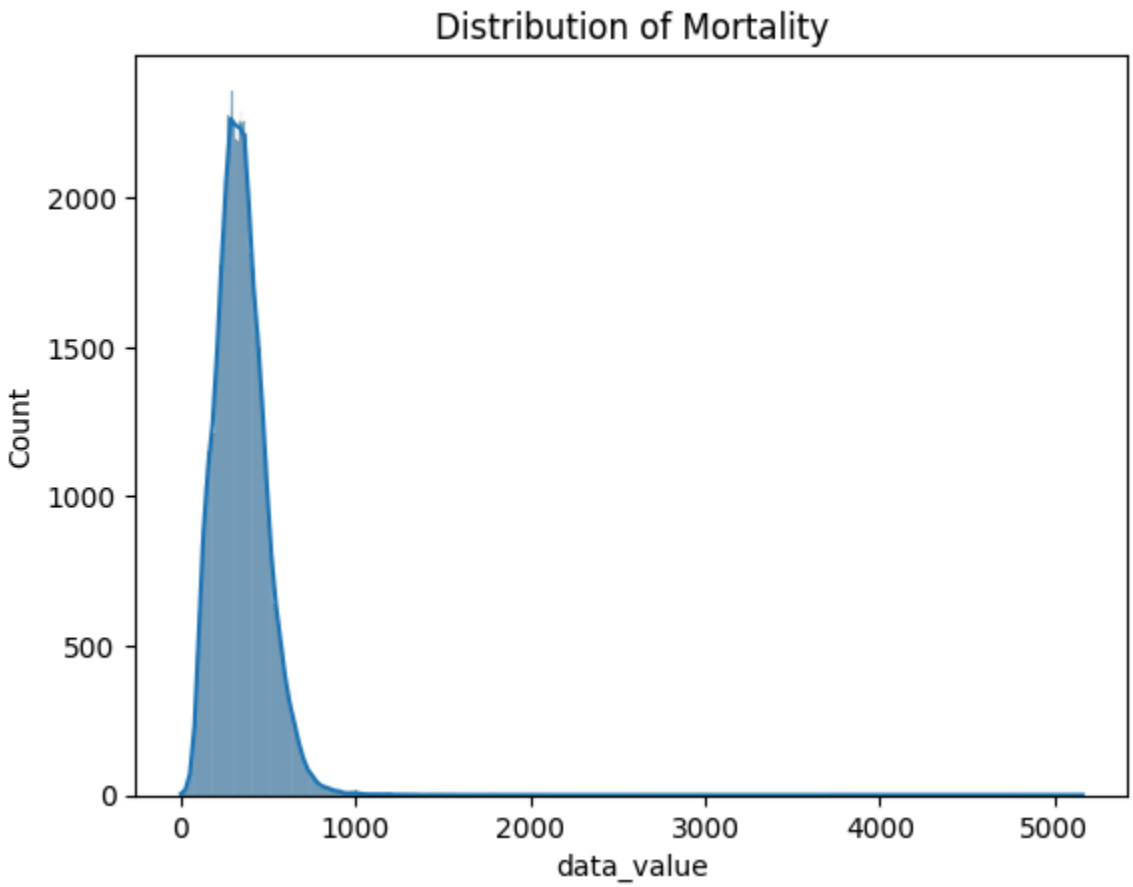
8. HANDLE TARGET COLUMN

In [9]:
```
df['data_value'] = pd.to_numeric(df['data_value'], errors='coerce')
df.dropna(subset=['data_value'], inplace=True)
```

9. EDA (VISUAL ANALYSIS)a) Distribution

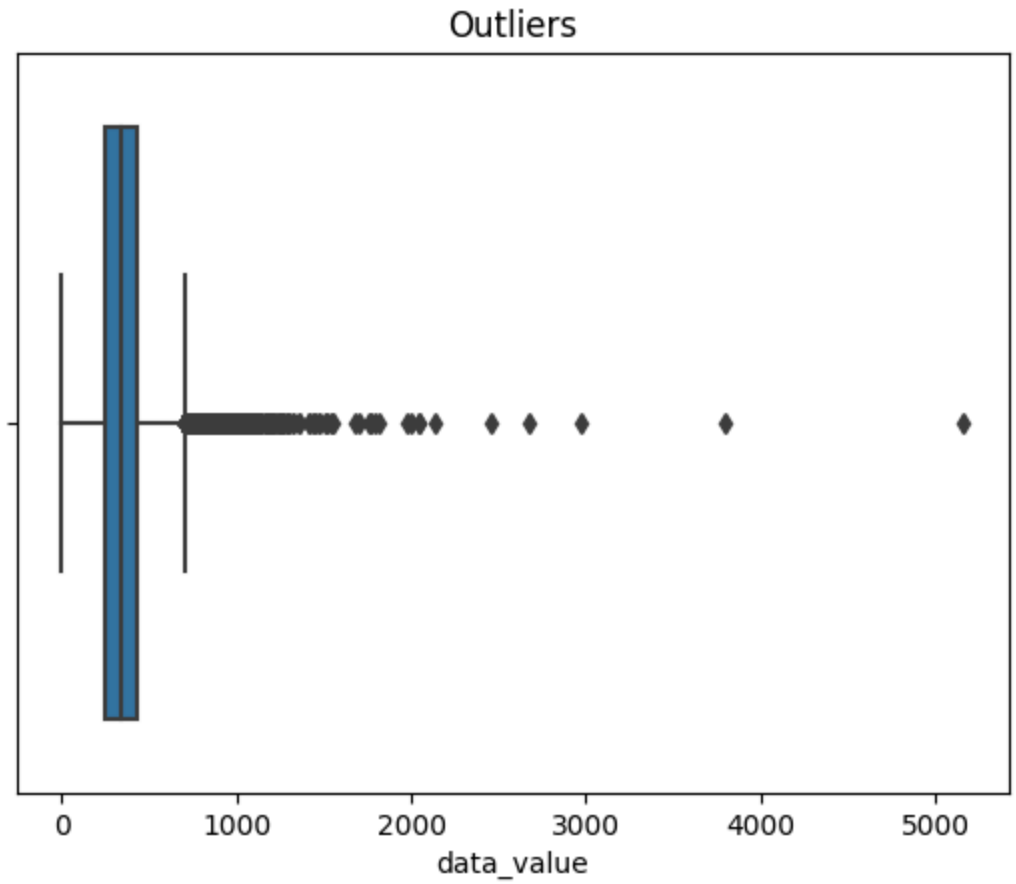
In [10]:
```
plt.figure()
sns.histplot(df['data_value'], kde=True)
plt.title("Distribution of Mortality")
plt.show()
```

```
D:\Program Files\Python310\lib\site-packages\seaborn\_oldcore.py:1119: FutureWa
rning: use_inf_as_na option is deprecated and will be removed in a future versi
on. Convert inf values to NaN before operating instead.
  with pd.option_context('mode.use_inf_as_na', True):
```



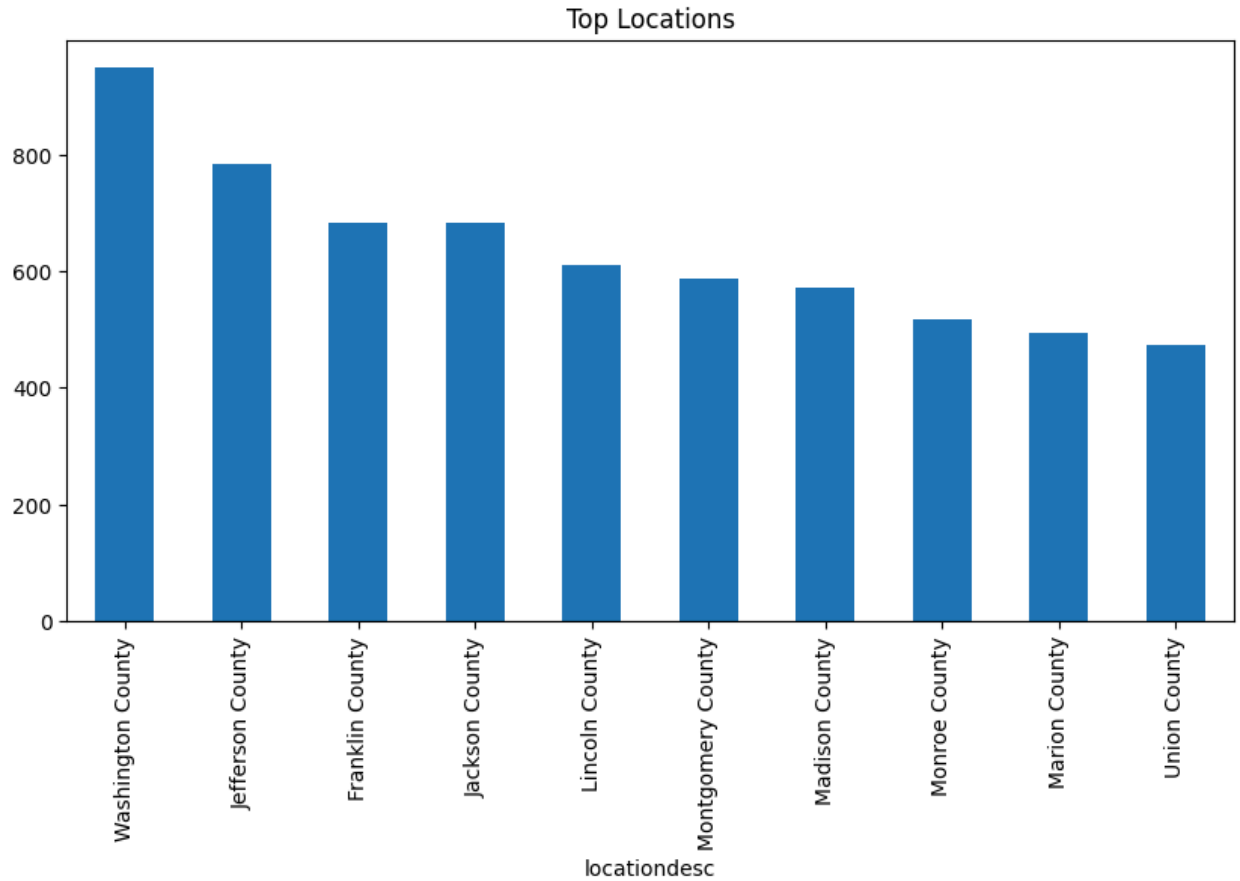
b) Boxplot (Outliers)

In [12]:
```
plt.figure()
sns.boxplot(x=df['data_value'])
plt.title("Outliers")
plt.show()
```



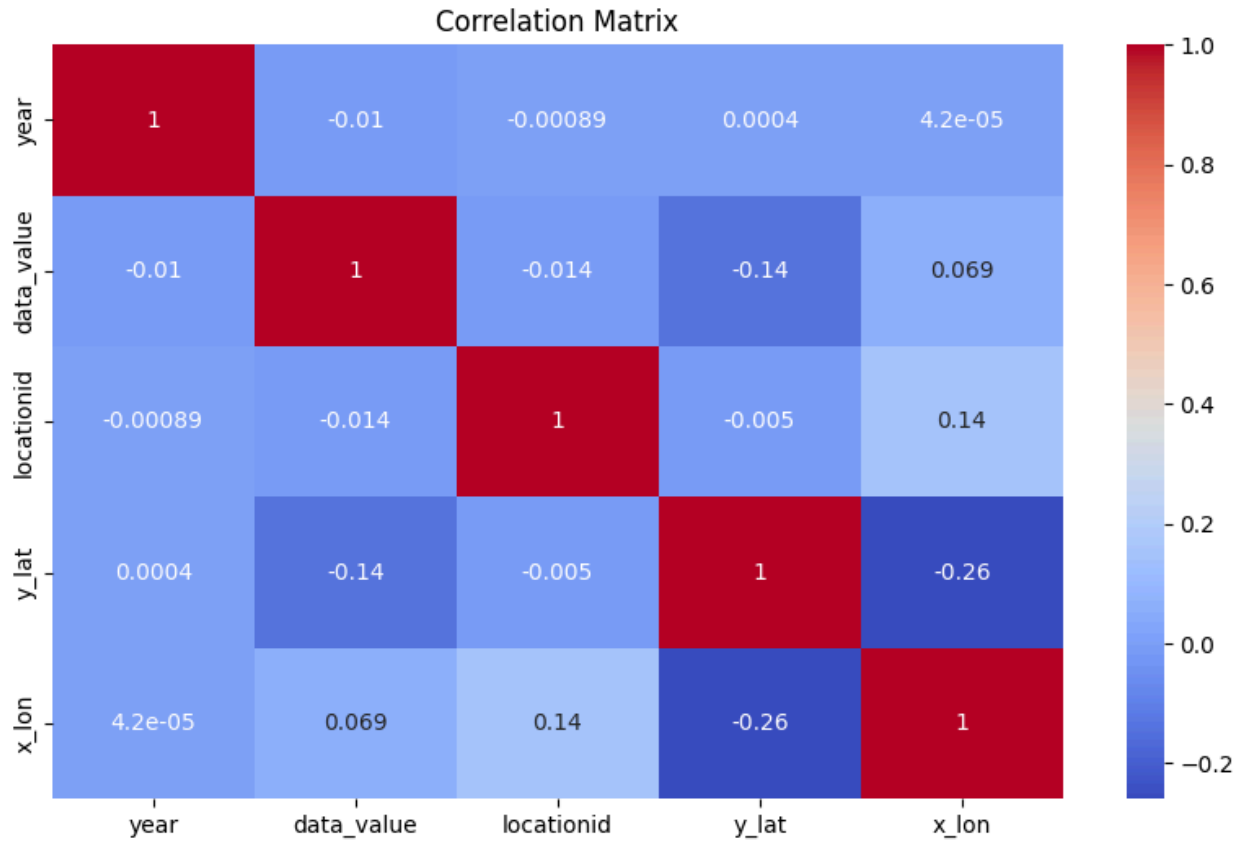
c) Top Locations

In [13]:
```
plt.figure(figsize=(10,5))
df['locationdesc'].value_counts().head(10).plot(kind='bar')
plt.title("Top Locations")
plt.show()
```



d) Correlation

In [14]:
```
plt.figure(figsize=(10,6))
sns.heatmap(df.select_dtypes(include=np.number).corr(), annot=True, cmap='cool
plt.title("Correlation Matrix")
plt.show()
```



10. OUTLIER REMOVAL

In [15]:
```
from scipy.stats import zscore
df = df[(np.abs(zscore(df['data_value'])) < 3)]
```

11. TARGET ENGINEERING

In [16]:
```
df['risk_category'] = pd.qcut(df['data_value'], q=3, labels=[0,1,2])
```

12. DEFINE FEATURES

In [17]:
```
X = df.drop(['data_value', 'risk_category'], axis=1)
y_reg = df['data_value']
y_clf = df['risk_category']
```

13. TRAIN-TEST SPLIT

In [18]:
```
X_train, X_test, y_train_r, y_test_r, y_train_c, y_test_c = train_test_split(
    X, y_reg, y_clf, test_size=0.2, random_state=42
)
```

14.ENCODING

In [19]:
```
X_train = pd.get_dummies(X_train, drop_first=True)
X_test = pd.get_dummies(X_test, drop_first=True)

X_train, X_test = X_train.align(X_test, join='left', axis=1, fill_value=0)
```

15. HANDLE MISSING VALUES (features X)

In [20]:
```
imputer = SimpleImputer(strategy='median')
```

```
X_train = imputer.fit_transform(X_train)
X_test = imputer.transform(X_test)
```

16. FEATURE SELECTION

In [21]:
```
selector = SelectKBest(score_func=f_regression, k=30)

X_train = selector.fit_transform(X_train, y_train_r)
X_test = selector.transform(X_test)
```

17. SCALING

In [22]:
```
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

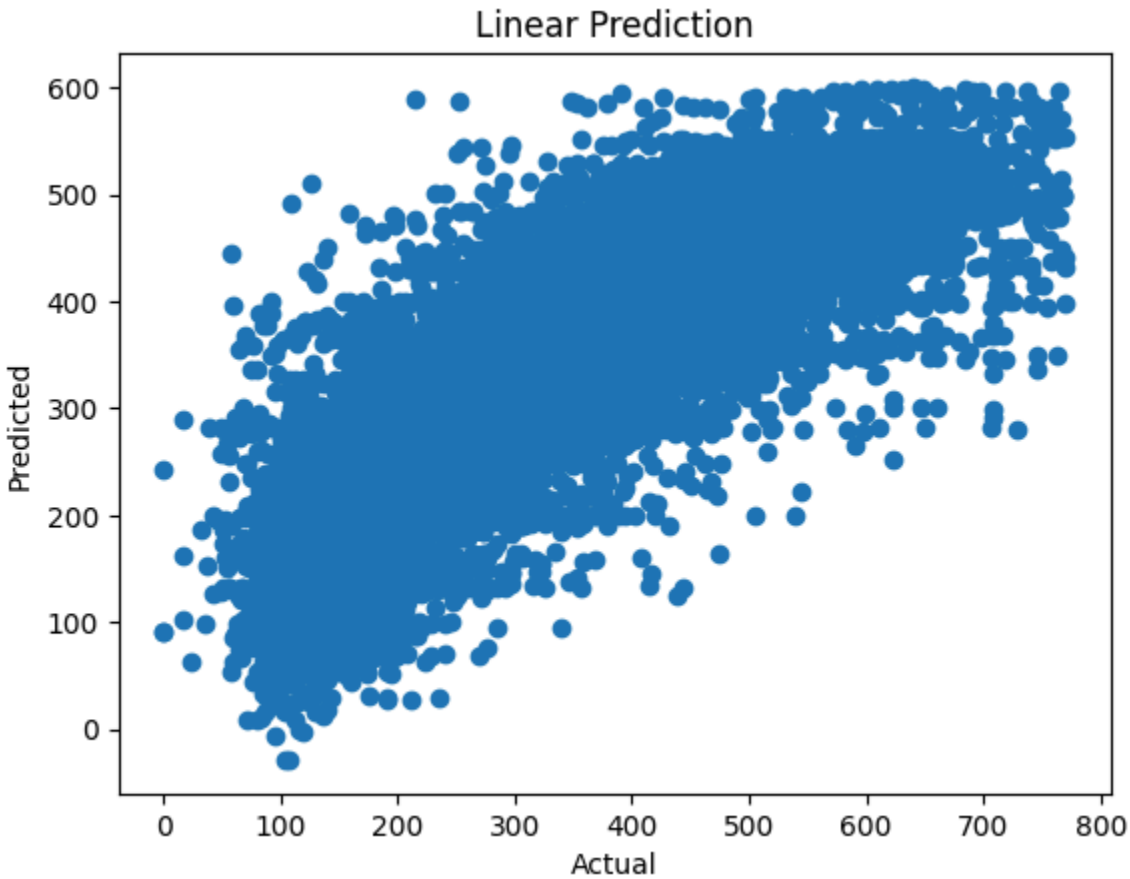
18. REGRESSION MODELS (5 MODELS)

In [23]:
```
reg_models = {
    "Linear": LinearRegression(),
    "Ridge": Ridge(),
    "Random Forest": RandomForestRegressor(),
    "Gradient Boosting": GradientBoostingRegressor(),
    "XGBoost": XGBRegressor()
}
```

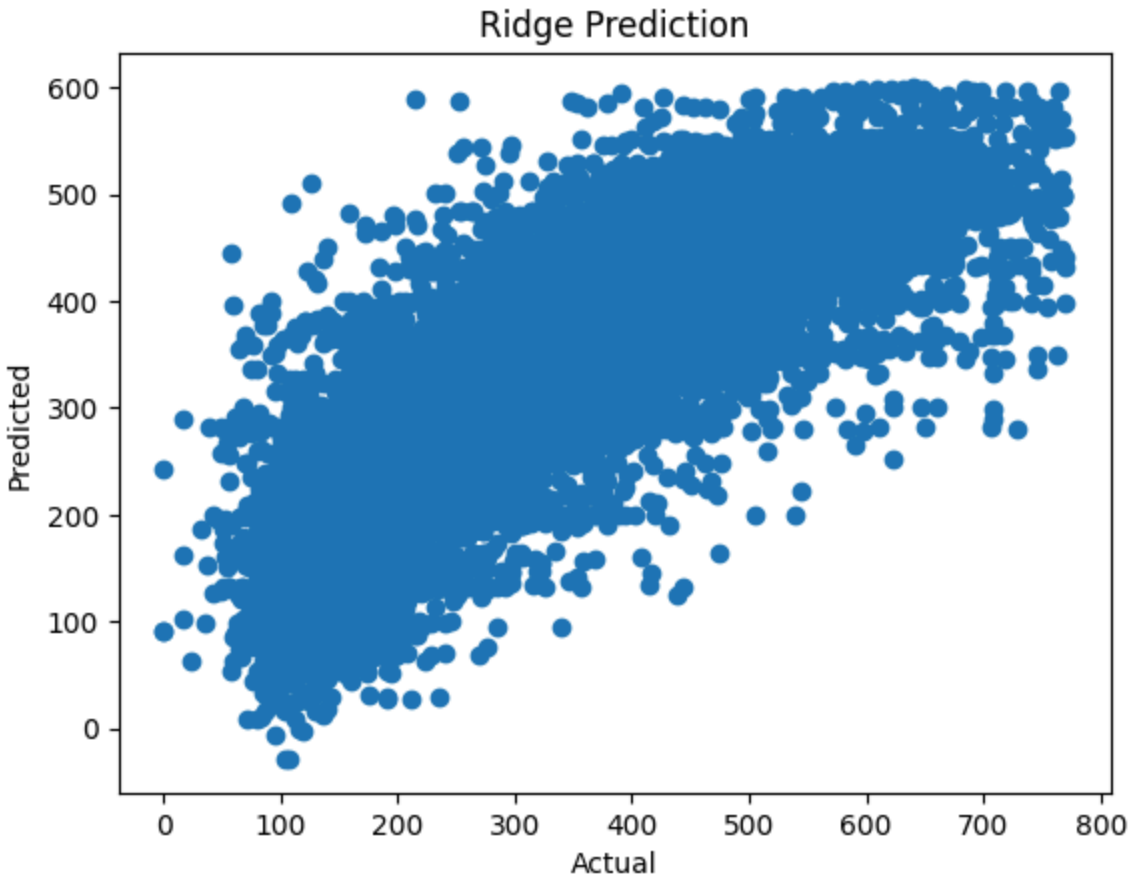
19. REGRESSION EVALUATE

In [24]:
```
reg_results = {}

for name, model in reg_models.items():

    model.fit(X_train, y_train_r)
    pred = model.predict(X_test)

    reg_results[name] = {
        "R2": r2_score(y_test_r, pred),
        "MAE": mean_absolute_error(y_test_r, pred),
        "RMSE": np.sqrt(mean_squared_error(y_test_r, pred))
    }

    print(f"\n{name} Results:")
    print("R2   :", reg_results[name]["R2"])
    print("MAE  :", reg_results[name]["MAE"])
    print("RMSE :", reg_results[name]["RMSE"])

    # Graph
    plt.figure()
    plt.scatter(y_test_r, pred)
    plt.title(f"{name} Prediction")
    plt.xlabel("Actual")
    plt.ylabel("Predicted")
    plt.show()
```

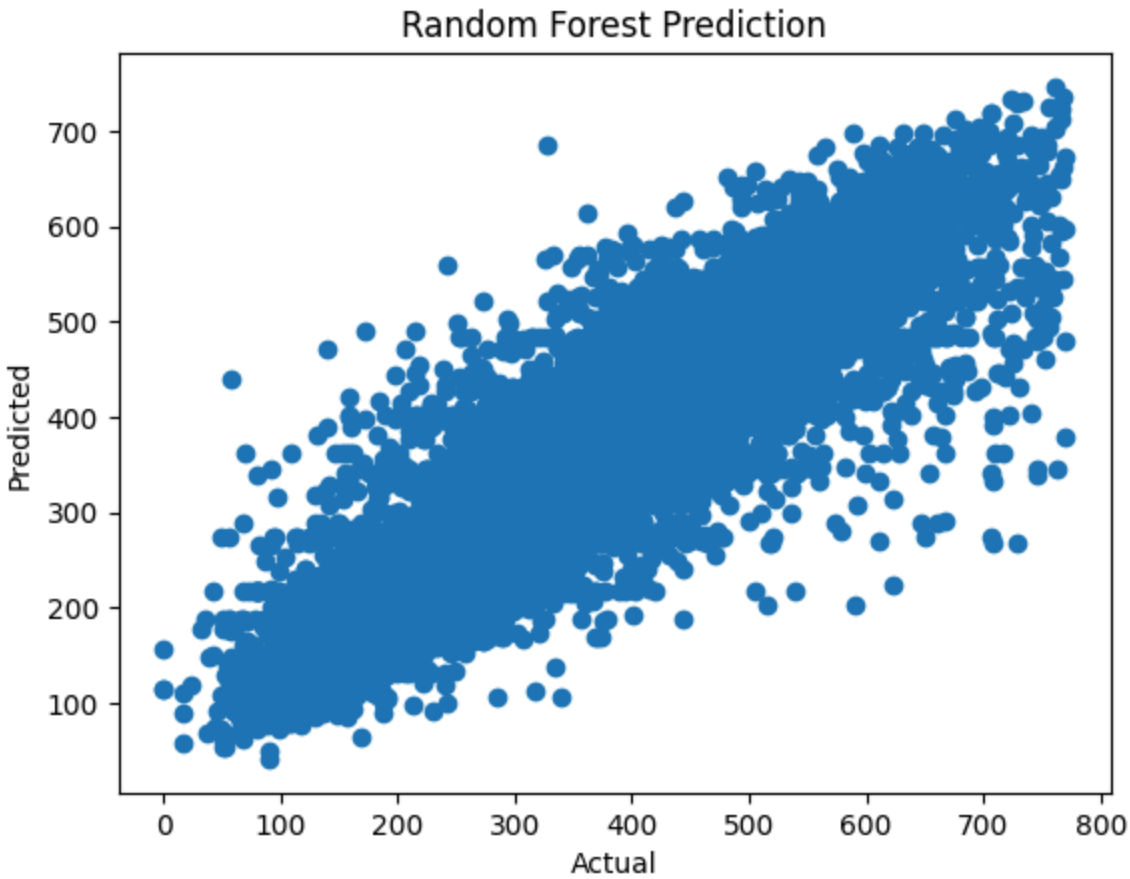
```
Linear Results:
R2   : 0.6495924326992815
MAE  : 58.86681183744311
RMSE : 78.8004811968318
```



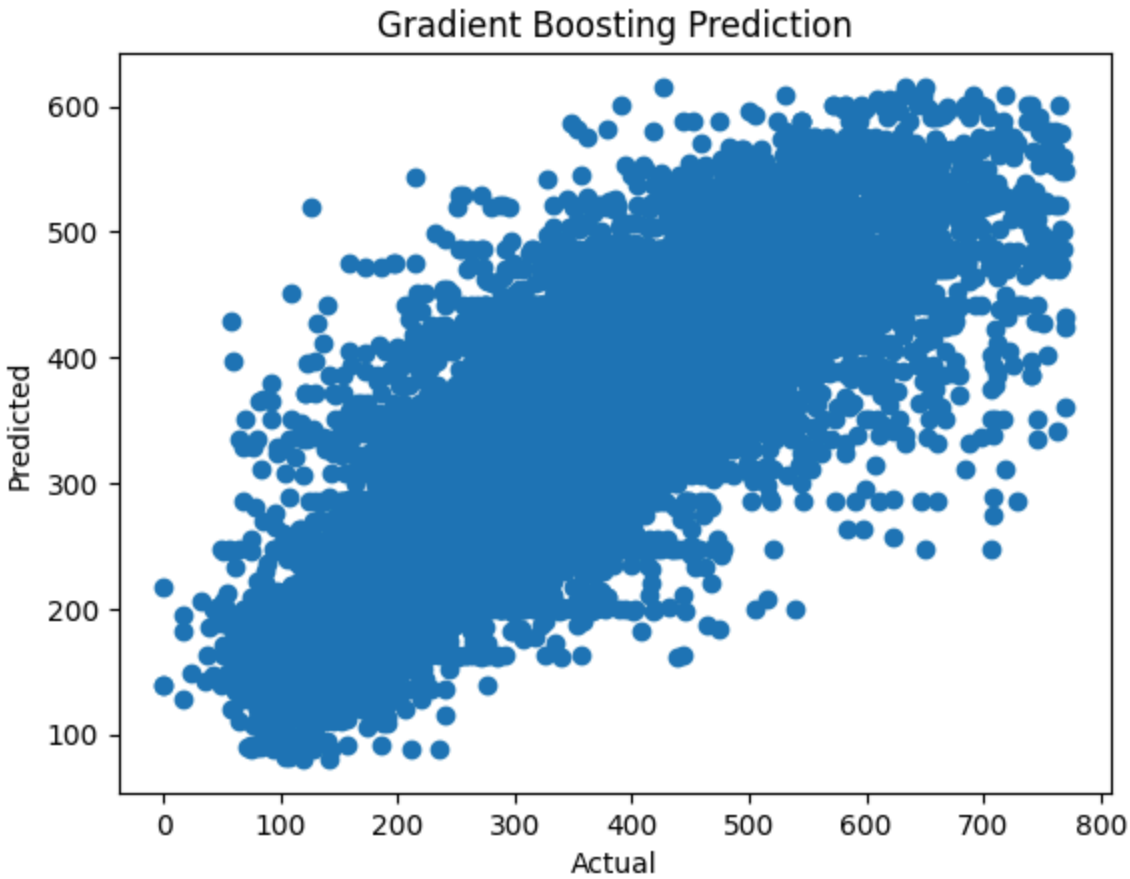

```
Ridge Results:
R2   : 0.6495922521547364
MAE  : 58.866819260977046
RMSE : 78.800501497471
```



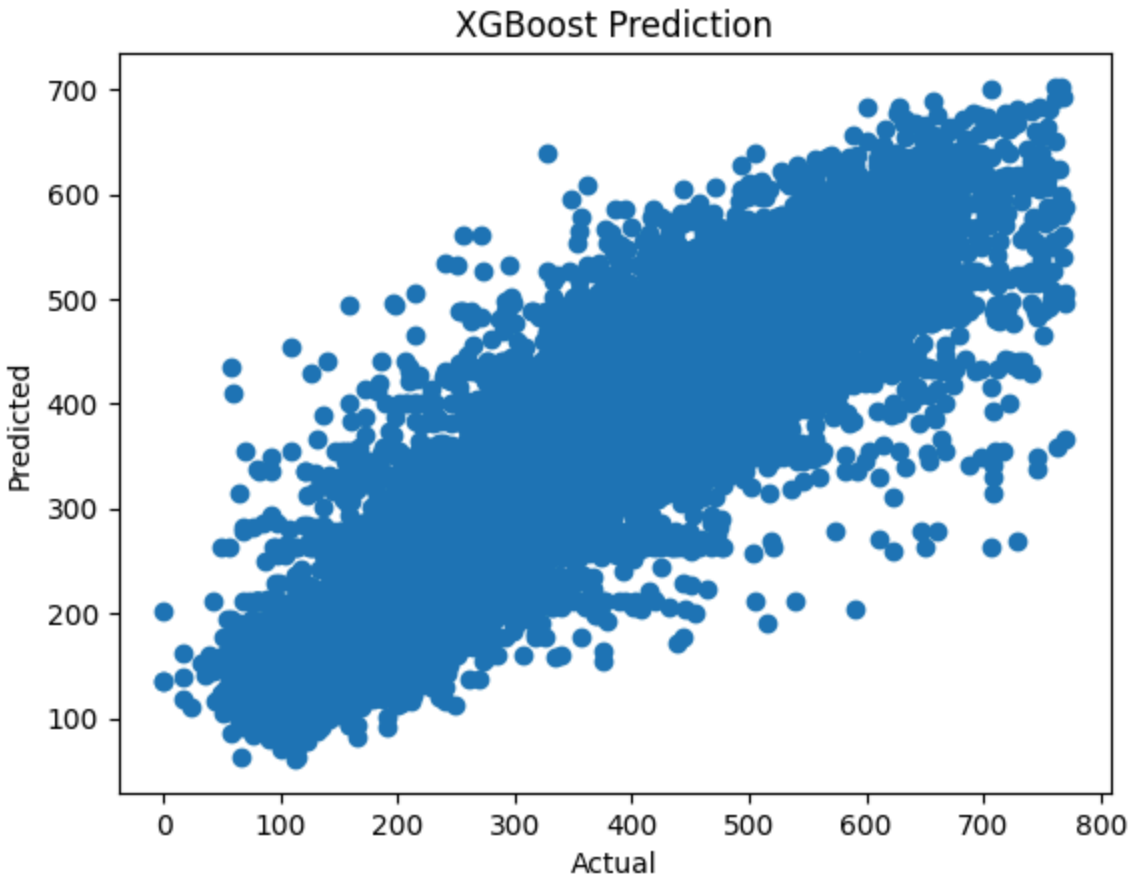

```
Random Forest Results:
R2   : 0.8347700041784942
MAE  : 34.74853913832037
RMSE : 54.111126740627455
```




```
Gradient Boosting Results:
R2   : 0.6862092531410848
MAE  : 54.939275304212295
RMSE : 74.5696659707661
```




```
XGBoost Results:
R2   : 0.7900567699220292
MAE  : 42.57264903828842
RMSE : 60.994836100815675
```



20. CLASSIFICATION MODELS (5 MODELS)

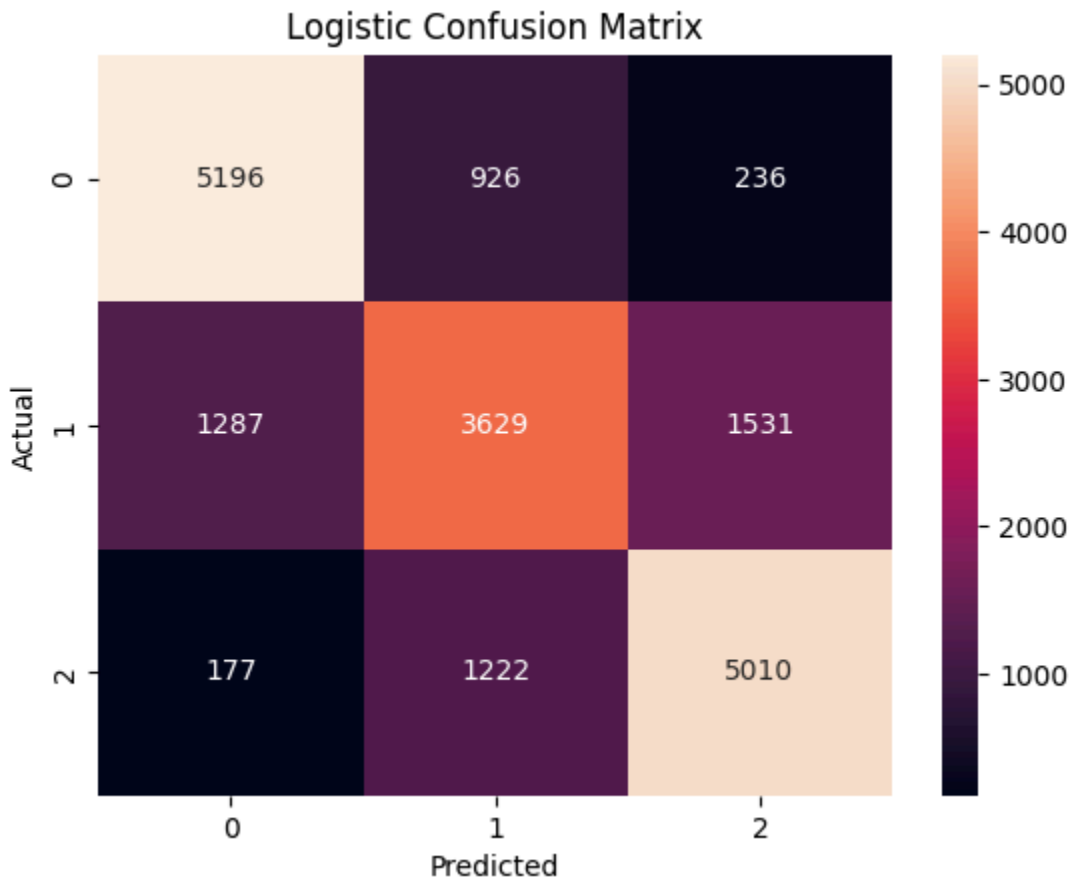
In [25]:
```
clf_models = {
    "Logistic": LogisticRegression(max_iter=500),
    "Random Forest": RandomForestClassifier(),
    "Gradient Boosting": GradientBoostingClassifier(),
    "KNN": KNeighborsClassifier(),
    "MLP": MLPClassifier(max_iter=500)
}
```

21. CLASSIFICATION EVALUATE

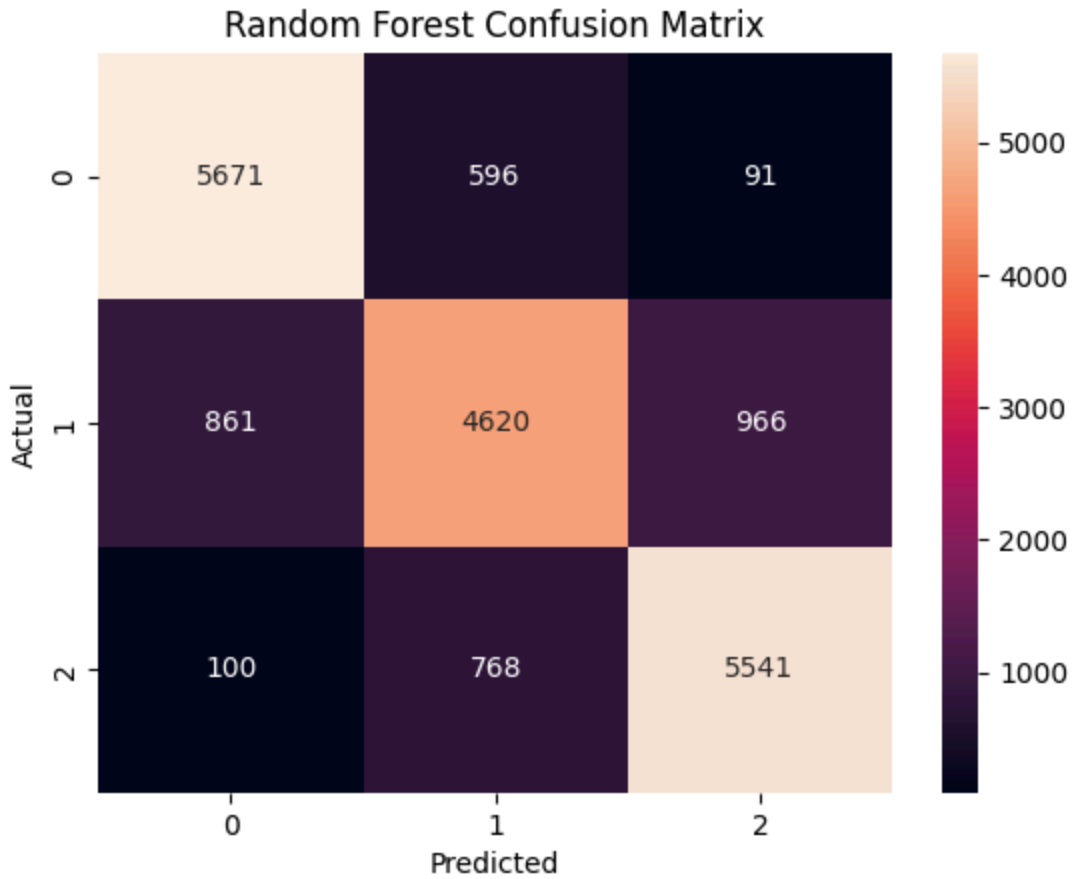
In [26]:
```
clf_results = {}

for name, model in clf_models.items():

    model.fit(X_train, y_train_c)
    pred = model.predict(X_test)

    clf_results[name] = {
        "Accuracy": accuracy_score(y_test_c, pred),
        "Precision": precision_score(y_test_c, pred, average='weighted'),
        "Recall": recall_score(y_test_c, pred, average='weighted'),
        "F1": f1_score(y_test_c, pred, average='weighted')
    }

    print(f"\n{name} Results:")
    print("Accuracy  :", clf_results[name]["Accuracy"])
```

```
    print("Precision :", clf_results[name]["Precision"])
    print("Recall    :", clf_results[name]["Recall"])
    print("F1 Score  :", clf_results[name]["F1"])

    # Confusion Matrix
    cm = confusion_matrix(y_test_c, pred)
    sns.heatmap(cm, annot=True, fmt='d')
    plt.title(f"{name} Confusion Matrix")
    plt.xlabel("Predicted")
    plt.ylabel("Actual")
    plt.show()
```

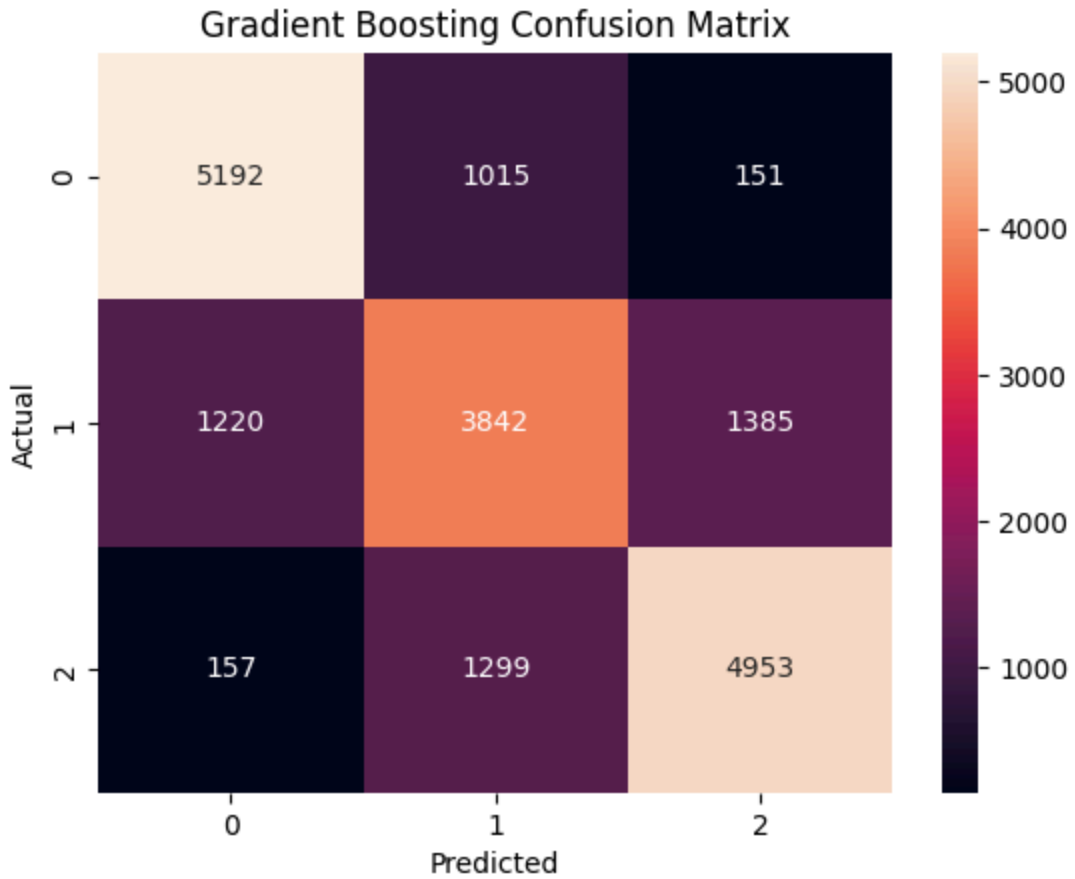
```
Logistic Results:
Accuracy  : 0.7200478817528885
Precision : 0.7155312317201546
Recall    : 0.7200478817528885
F1 Score  : 0.7168494354409031
```



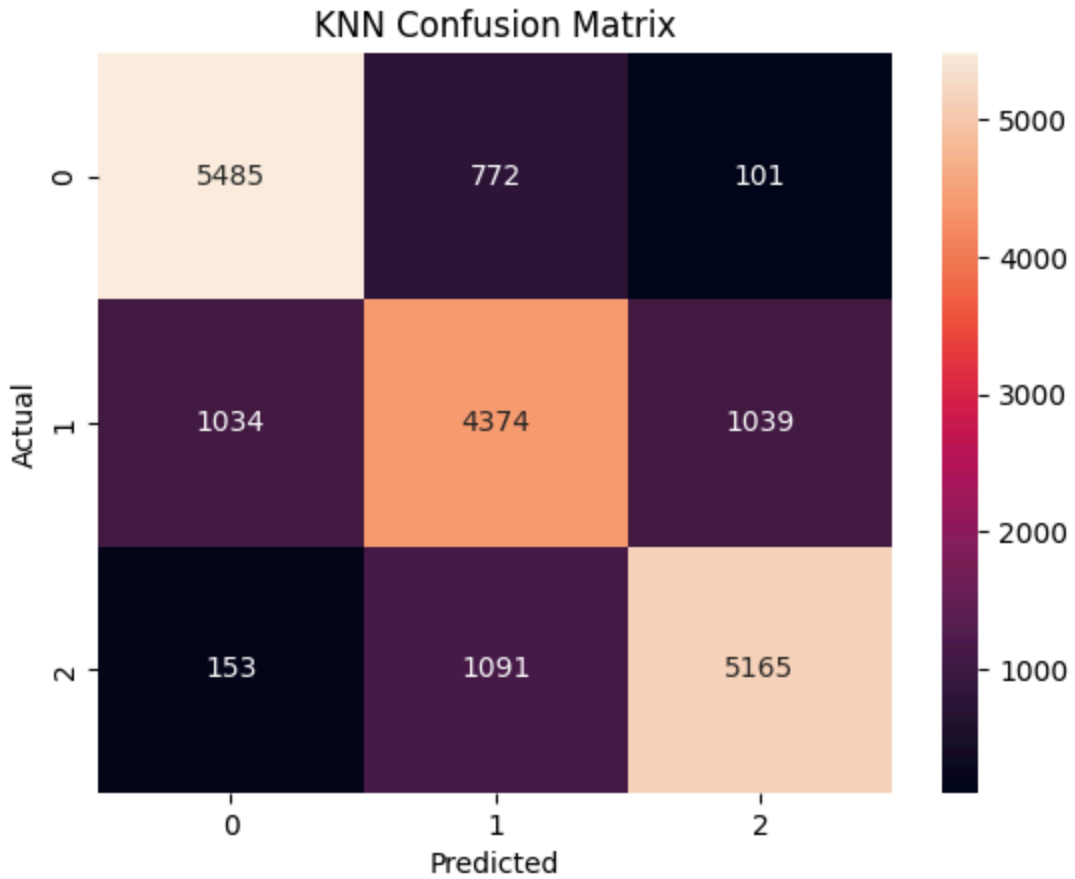

```
Random Forest Results:
Accuracy  : 0.823982512751119
Precision : 0.8221320185935442
Recall    : 0.823982512751119
F1 Score  : 0.8225221788776147
```



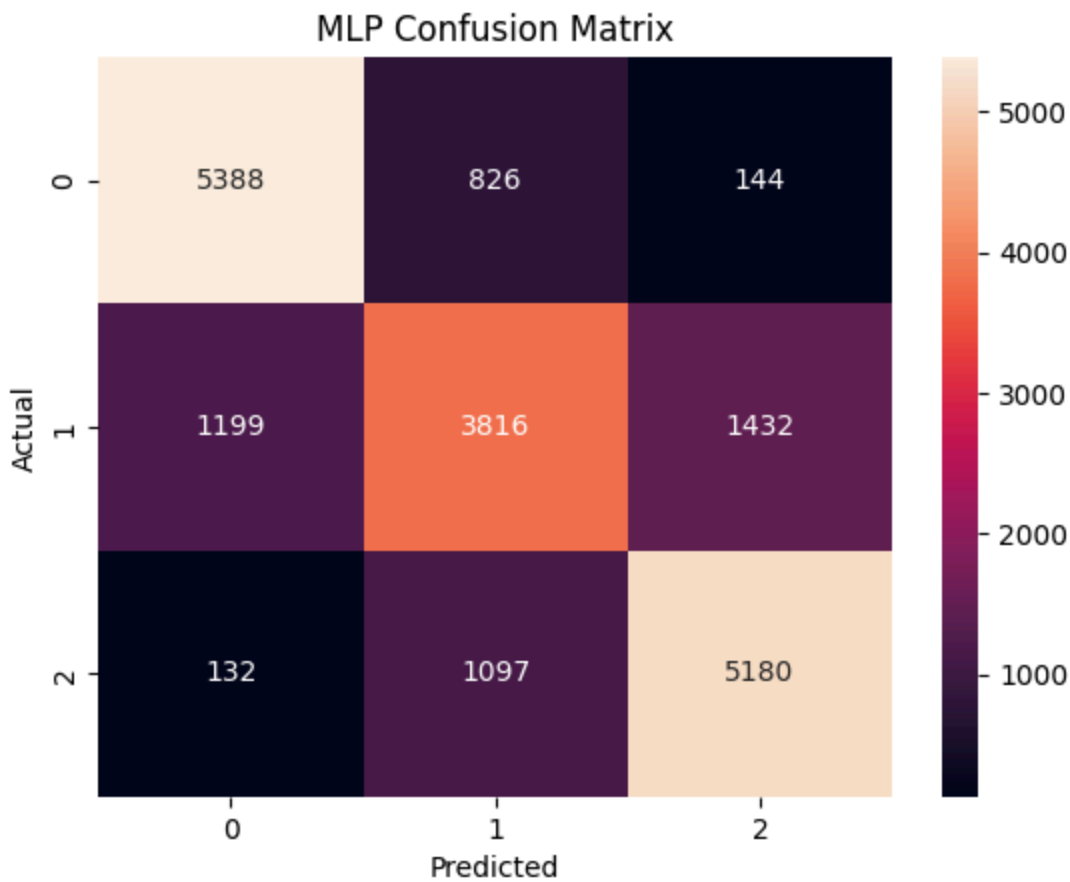

```
Gradient Boosting Results:
Accuracy  : 0.7279587800562091
Precision : 0.725553330466325
Recall    : 0.7279587800562091
F1 Score  : 0.7265662384560805
```




```
KNN Results:
Accuracy  : 0.7819298428229416
Precision : 0.7805939217837886
Recall    : 0.7819298428229416
F1 Score  : 0.781018418298301
```

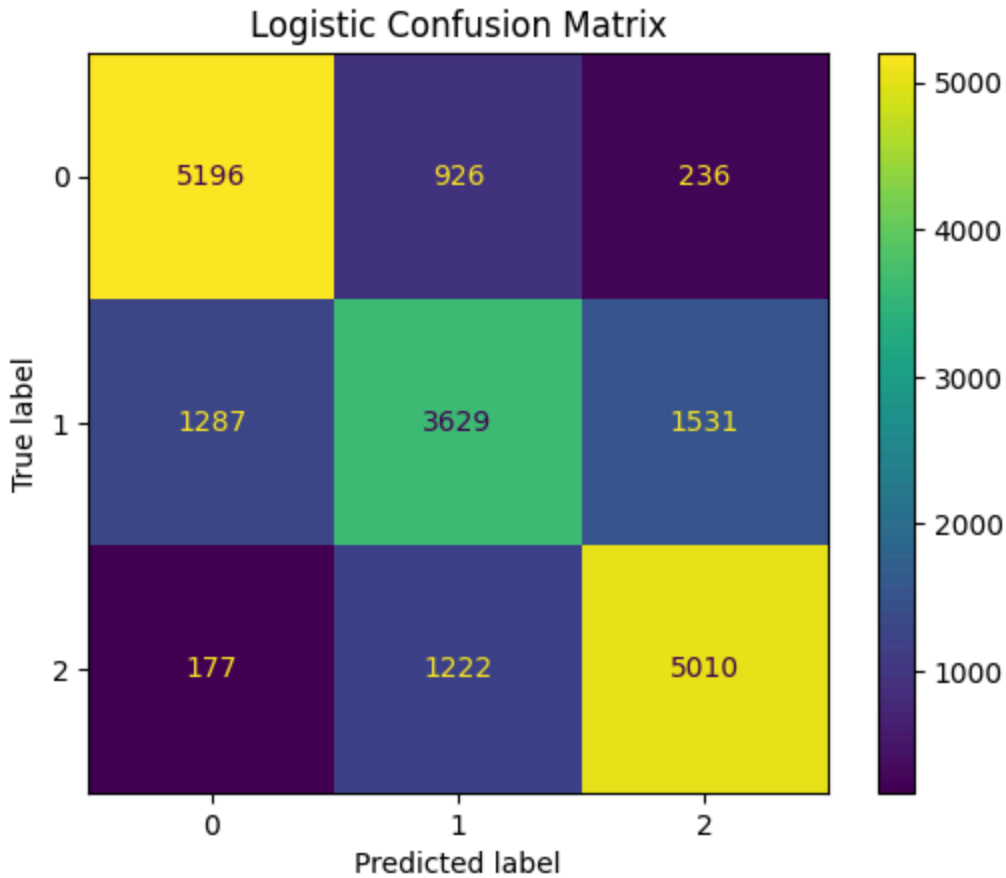



```
MLP Results:
Accuracy  : 0.7486207973352763
Precision : 0.7442086362119852
Recall    : 0.7486207973352763
F1 Score  : 0.7453125159780086
```

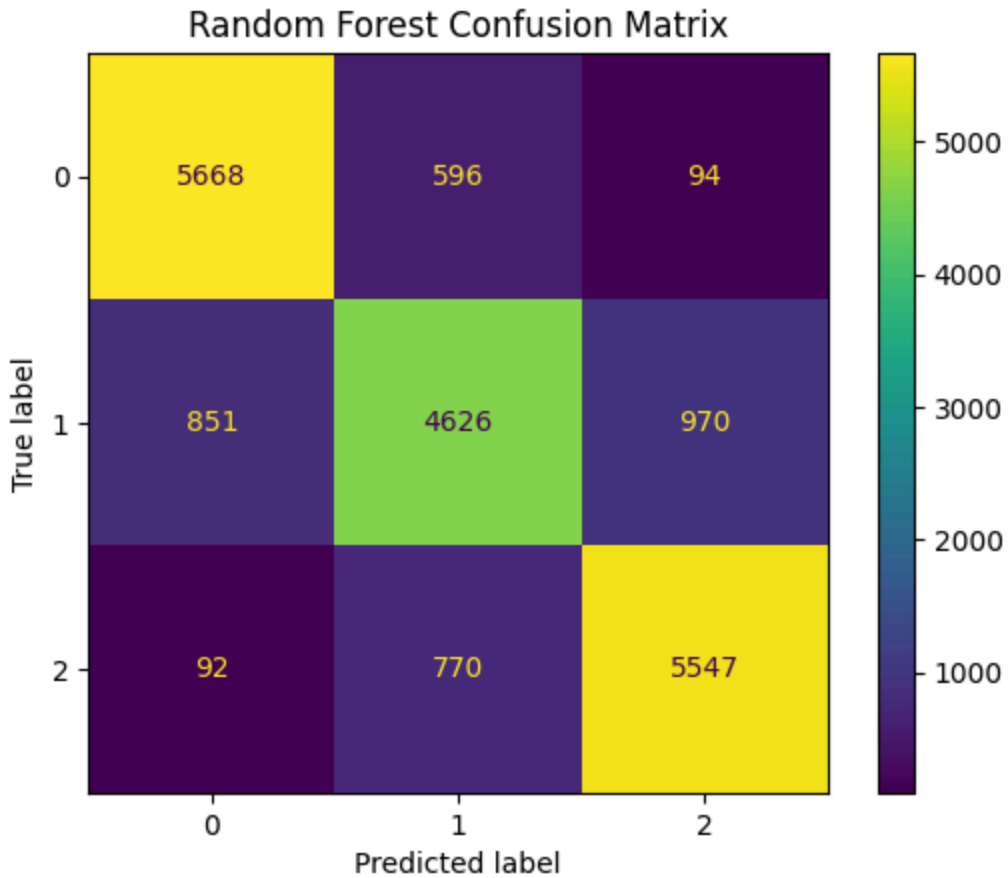


In [27]:
```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

for name, model in clf_models.items():

    model.fit(X_train, y_train_c)
    pred = model.predict(X_test)

    cm = confusion_matrix(y_test_c, pred)

    print(f"\n{name} Confusion Matrix:\n", cm)

    # Visual display
    disp = ConfusionMatrixDisplay(confusion_matrix=cm)
    disp.plot()
    plt.title(f"{name} Confusion Matrix")
    plt.show()
```

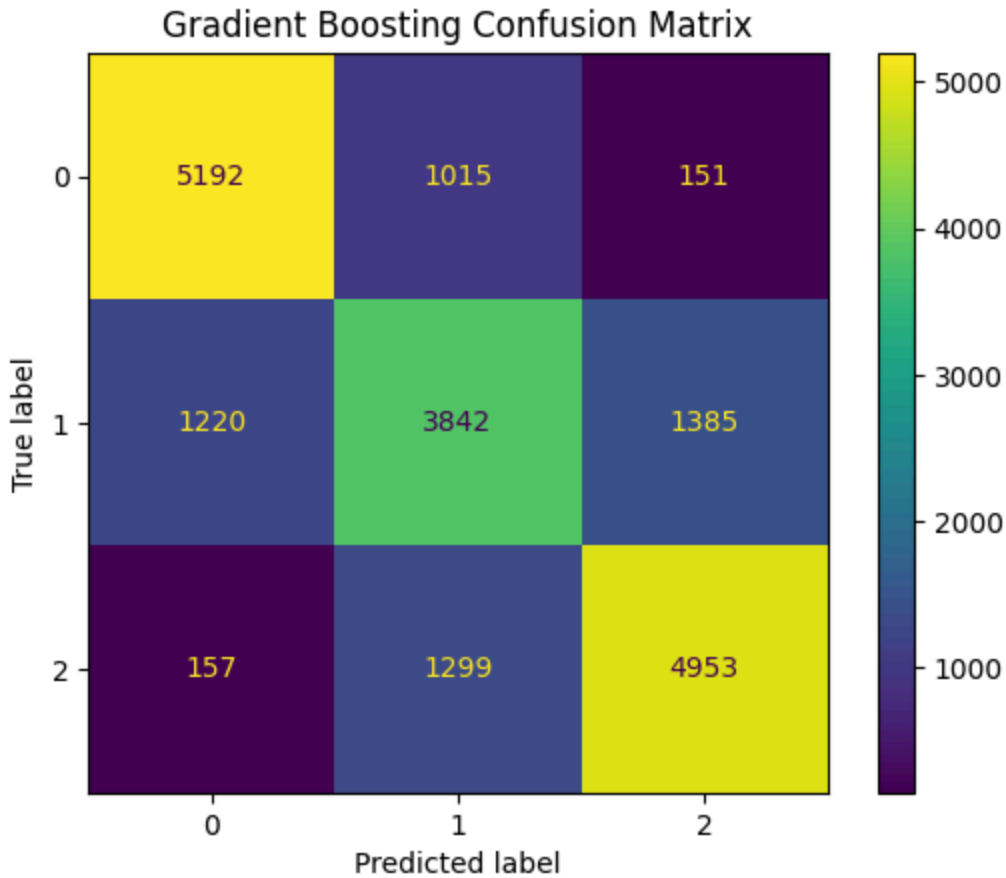
```
Logistic Confusion Matrix:
 [[5196  926  236]
 [1287 3629 1531]
 [ 177 1222 5010]]
```



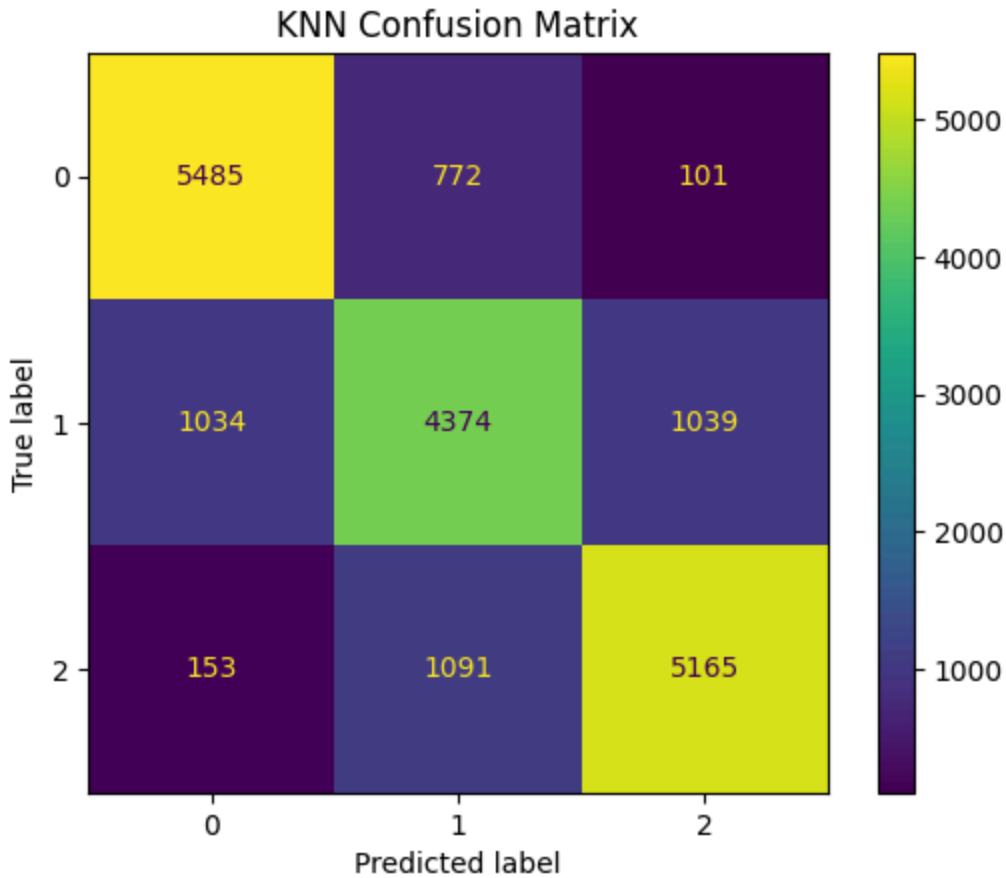

```
Random Forest Confusion Matrix:
 [[5668  596   94]
 [ 851 4626  970]
 [  92  770 5547]]
```



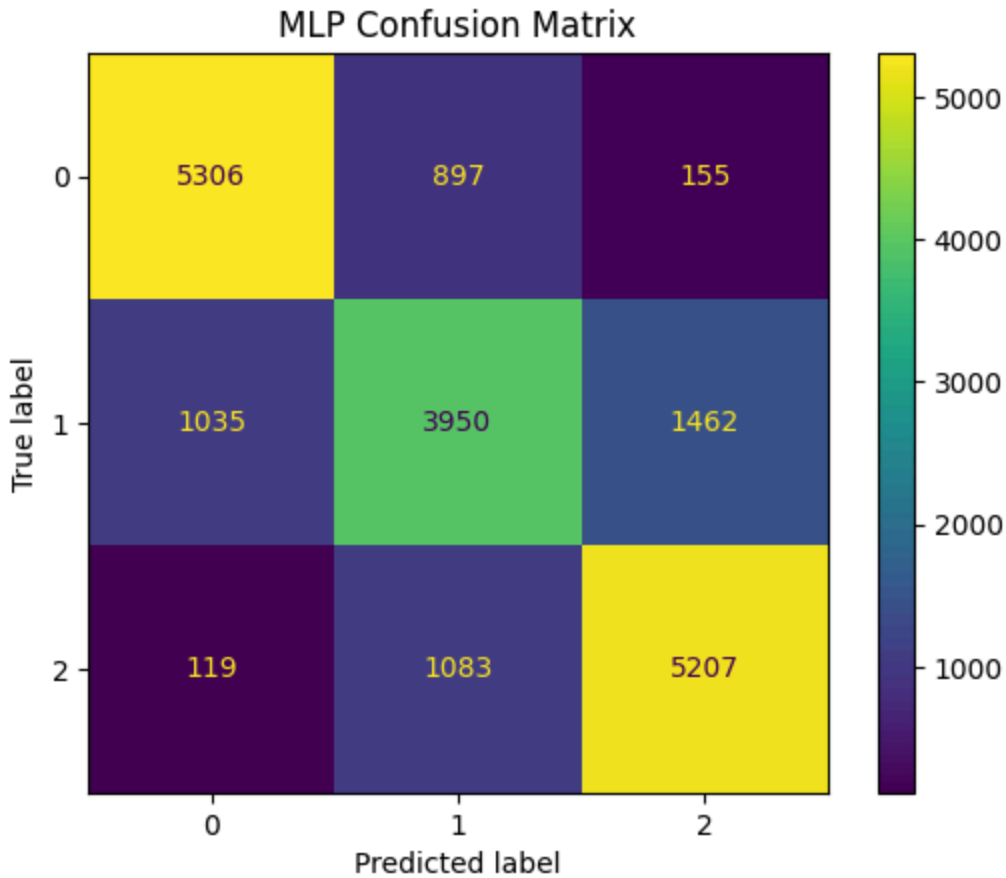

```
Gradient Boosting Confusion Matrix:
 [[5192 1015  151]
 [1220 3842 1385]
 [ 157 1299 4953]]
```




```
KNN Confusion Matrix:
 [[5485  772  101]
 [1034 4374 1039]
 [ 153 1091 5165]]
```




```
MLP Confusion Matrix:
 [[5306  897  155]
 [1035 3950 1462]
 [ 119 1083 5207]]
```



In [28]:

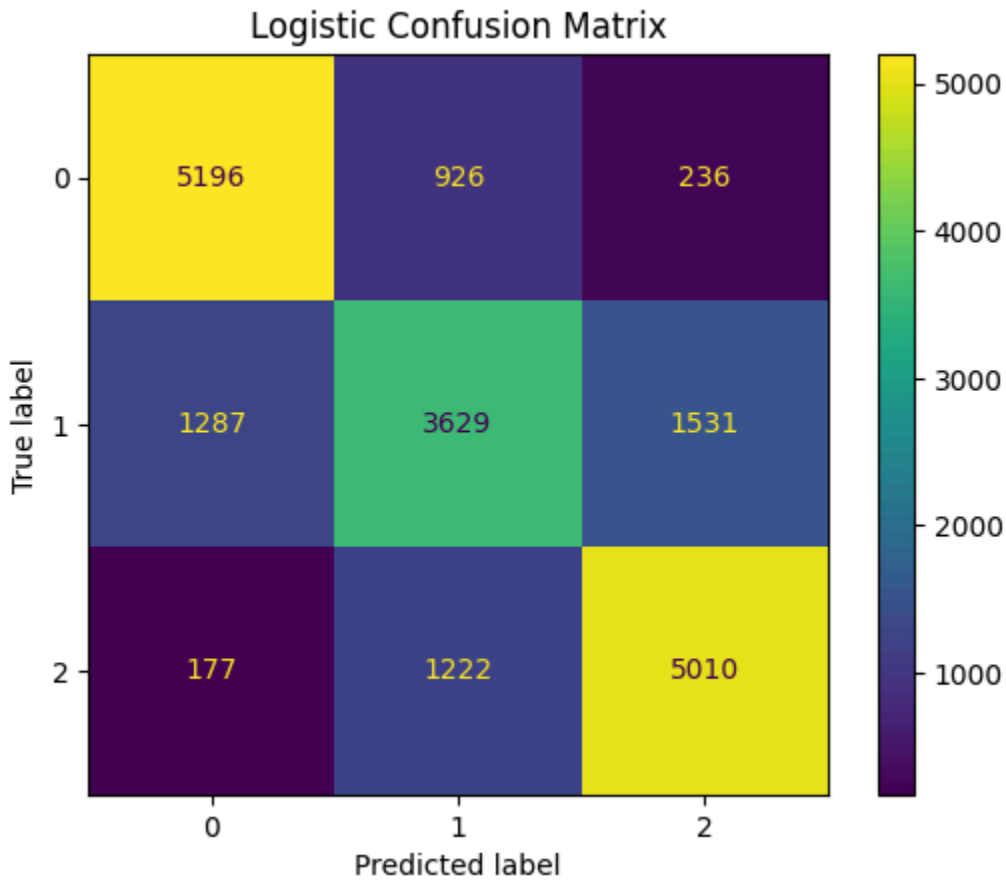
```
import numpy as np

cm = confusion_matrix(y_test_c, pred)

if cm.shape == (2, 2):
    TN, FP, FN, TP = cm.ravel()

    accuracy = (TP + TN) / (TP + TN + FP + FN)
    precision = TP / (TP + FP)
    recall = TP / (TP + FN)
    f1 = 2 * precision * recall / (precision + recall)

    print("Accuracy:", accuracy)
    print("Precision:", precision)
    print("Recall:", recall)
    print("F1:", f1)

else:
    from sklearn.metrics import accuracy_score, precision_score, recall_score,

    print("Accuracy:", accuracy_score(y_test_c, pred))
    print("Precision:", precision_score(y_test_c,pred, average='weighted'))
    print("Recall:", recall_score(y_test_c, pred, average='weighted'))
    print("F1:", f1_score(y_test_c, pred, average='weighted'))
```

```
Accuracy: 0.7527323826376601
Precision: 0.7498144369699342
Recall: 0.7527323826376601
F1: 0.7506232987439659
```

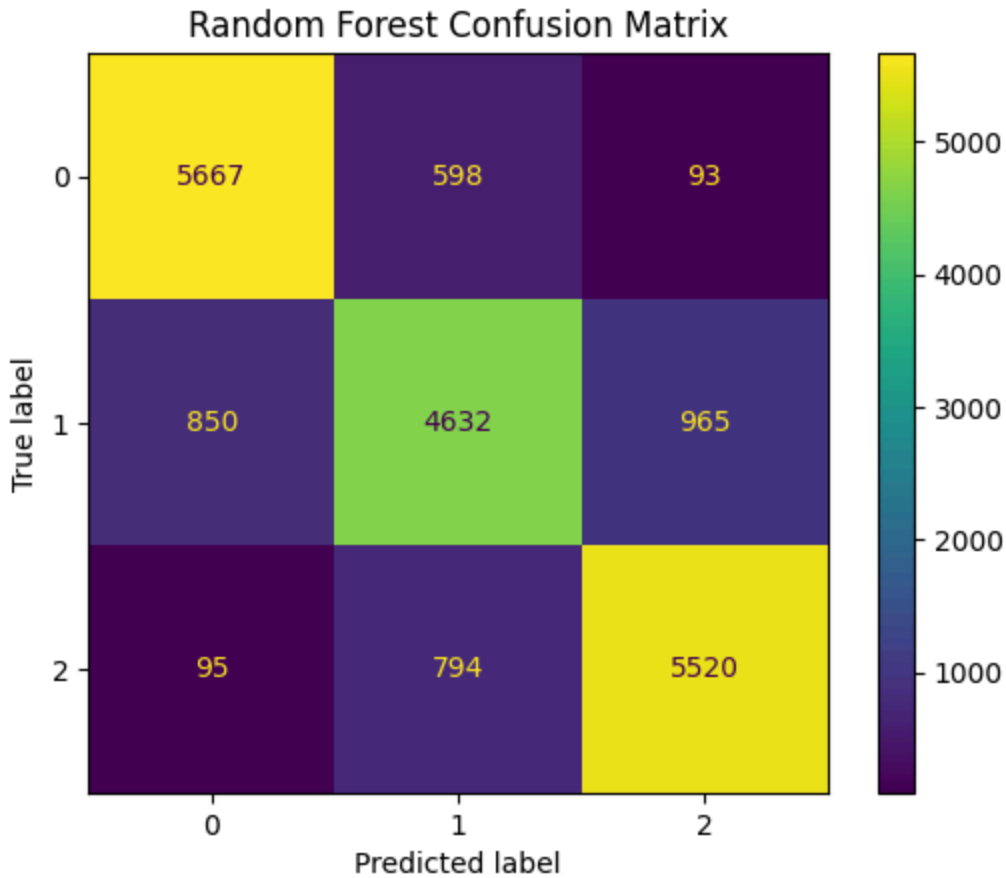
In [29]:

```python
from sklearn.metrics import (
    confusion_matrix,
    ConfusionMatrixDisplay,
    classification_report,
    roc_auc_score,
    roc_curve
)
import matplotlib.pyplot as plt

for name, model in clf_models.items():

    model.fit(X_train, y_train_c)
    pred = model.predict(X_test)

    print(f"\n===== {name} =====")

    #  🔹 Confusion Matrix
    cm = confusion_matrix(y_test_c, pred)
    print("Confusion Matrix:\n", cm)

    disp = ConfusionMatrixDisplay(confusion_matrix=cm)
    disp.plot()
    plt.title(f"{name} Confusion Matrix")
    plt.show()

    #  🔹 Classification Report
    print("\nClassification Report:\n")
    print(classification_report(y_test_c, pred))

    #  🔹 ROC-AUC (only for binary classification)
    if len(set(y_test_c)) == 2:
        if hasattr(model, "predict_proba"):
            prob = model.predict_proba(X_test)[:, 1]
        else:
            prob = model.decision_function(X_test)

        auc = roc_auc_score(y_test_c, prob)
        print("ROC-AUC Score:", auc)

        fpr, tpr, _ = roc_curve(y_test_c, prob)
        plt.plot(fpr, tpr)
        plt.plot([0,1],[0,1],'--')
        plt.title(f"{name} ROC Curve")
        plt.xlabel("False Positive Rate")
        plt.ylabel("True Positive Rate")
        plt.show()
```

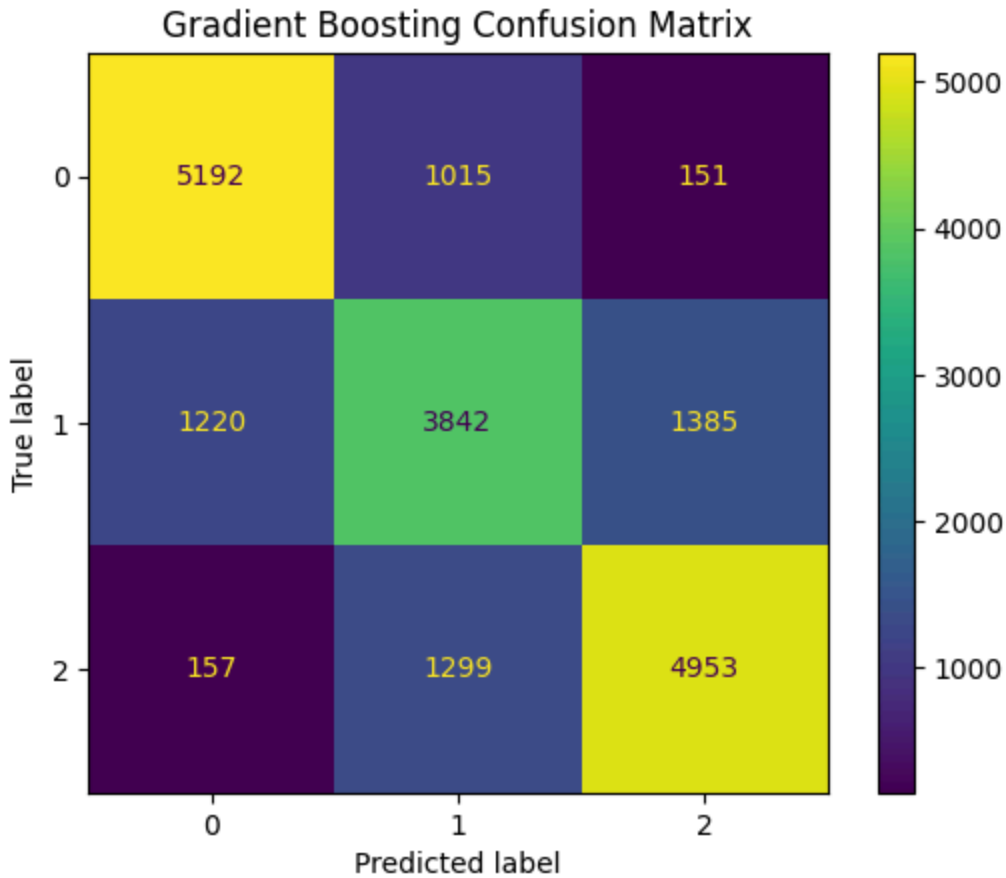
```
===== Logistic =====
Confusion Matrix:
 [[5196  926  236]
 [1287 3629 1531]
 [ 177 1222 5010]]
```



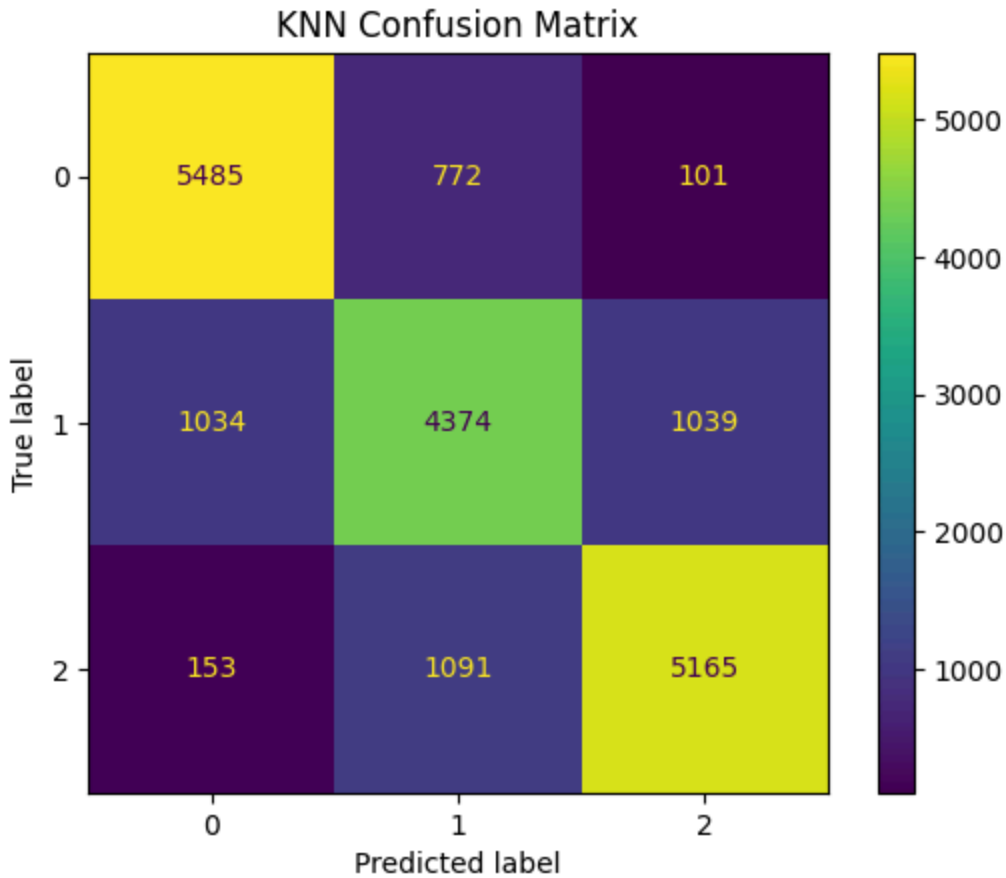

```
Classification Report:

              precision    recall  f1-score   support

           0       0.78      0.82      0.80      6358
           1       0.63      0.56      0.59      6447
           2       0.74      0.78      0.76      6409

    accuracy                           0.72     19214
   macro avg       0.72      0.72      0.72     19214
weighted avg       0.72      0.72      0.72     19214


===== Random Forest =====
Confusion Matrix:
 [[5667  598   93]
 [ 850 4632  965]
 [  95  794 5520]]
```



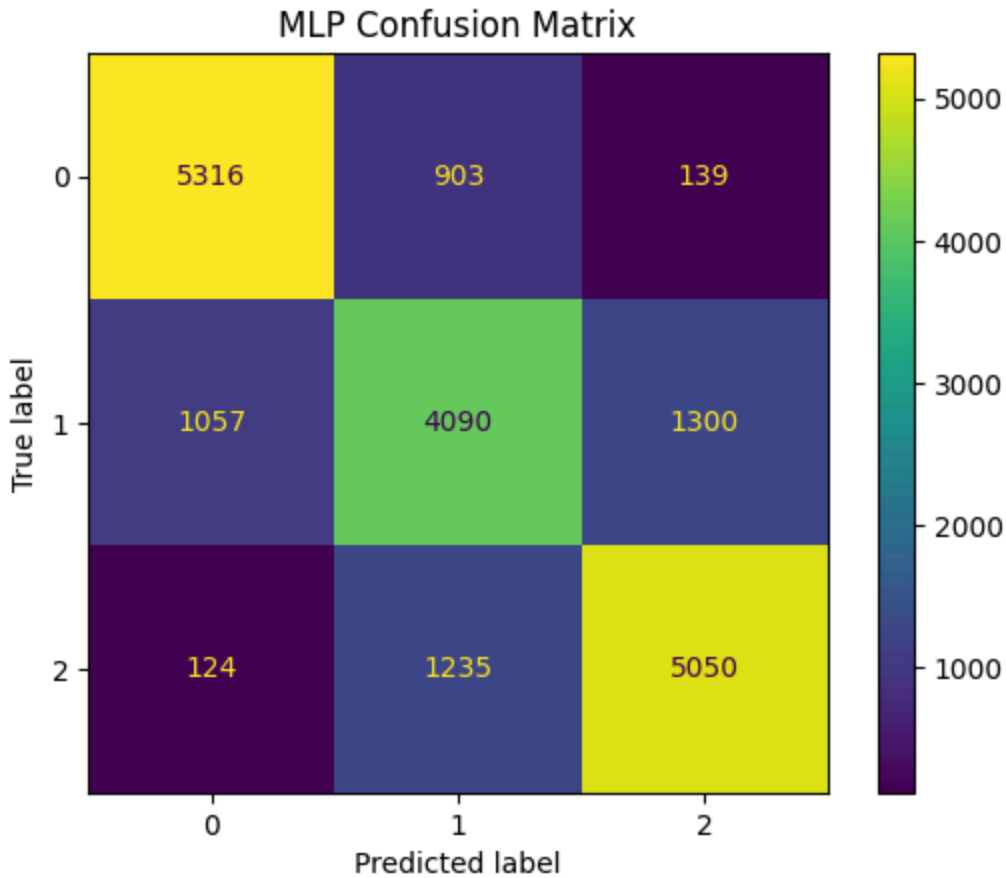

```
Classification Report:

              precision    recall  f1-score   support

           0       0.86      0.89      0.87      6358
           1       0.77      0.72      0.74      6447
           2       0.84      0.86      0.85      6409

    accuracy                           0.82     19214
   macro avg       0.82      0.82      0.82     19214
weighted avg       0.82      0.82      0.82     19214


===== Gradient Boosting =====
Confusion Matrix:
 [[5192 1015  151]
 [1220 3842 1385]
 [ 157 1299 4953]]
```




```
Classification Report:

              precision    recall  f1-score   support

           0       0.79      0.82      0.80      6358
           1       0.62      0.60      0.61      6447
           2       0.76      0.77      0.77      6409

    accuracy                           0.73     19214
   macro avg       0.73      0.73      0.73     19214
weighted avg       0.73      0.73      0.73     19214


===== KNN =====
Confusion Matrix:
 [[5485  772  101]
 [1034 4374 1039]
 [ 153 1091 5165]]
```




```
Classification Report:

              precision    recall  f1-score   support

           0       0.82      0.86      0.84      6358
           1       0.70      0.68      0.69      6447
           2       0.82      0.81      0.81      6409

    accuracy                           0.78     19214
   macro avg       0.78      0.78      0.78     19214
weighted avg       0.78      0.78      0.78     19214


===== MLP =====
Confusion Matrix:
 [[5316  903  139]
 [1057 4090 1300]
 [ 124 1235 5050]]
```




```
Classification Report:

              precision    recall  f1-score   support

           0       0.82      0.84      0.83      6358
           1       0.66      0.63      0.65      6447
           2       0.78      0.79      0.78      6409

    accuracy                           0.75     19214
   macro avg       0.75      0.75      0.75     19214
weighted avg       0.75      0.75      0.75     19214
```

In [30]:

```python
clf_summary = {}

for name, model in clf_models.items():

    model.fit(X_train, y_train_c)
    pred = model.predict(X_test)

    clf_summary[name] = {
        "Accuracy": accuracy_score(y_test_c, pred),
        "Precision": precision_score(y_test_c, pred, average='weighted'),
        "Recall": recall_score(y_test_c, pred, average='weighted'),
        "F1 Score": f1_score(y_test_c, pred, average='weighted')
    }
```

```
clf_df = pd.DataFrame(clf_summary).T

print("\nFinal Model Comparison:\n")
print(clf_df)
```

```
Final Model Comparison:

                   Accuracy  Precision    Recall  F1 Score
Logistic           0.720048   0.715531  0.720048  0.716849
Random Forest      0.824035   0.822305  0.824035  0.822736
Gradient Boosting  0.727959   0.725553  0.727959  0.726566
KNN                0.781930   0.780594  0.781930  0.781018
MLP                0.749870   0.744992  0.749870  0.745424
```

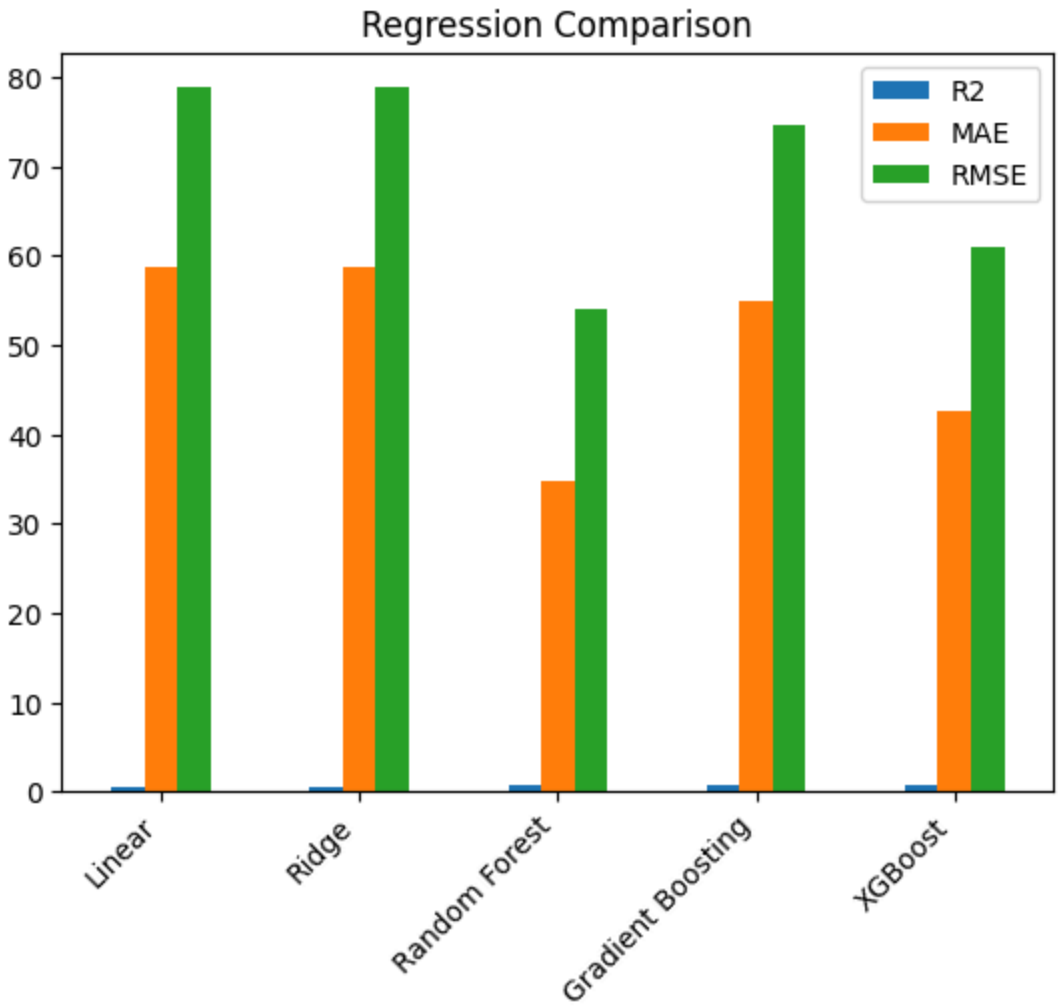
22. COMPARISON TABLES

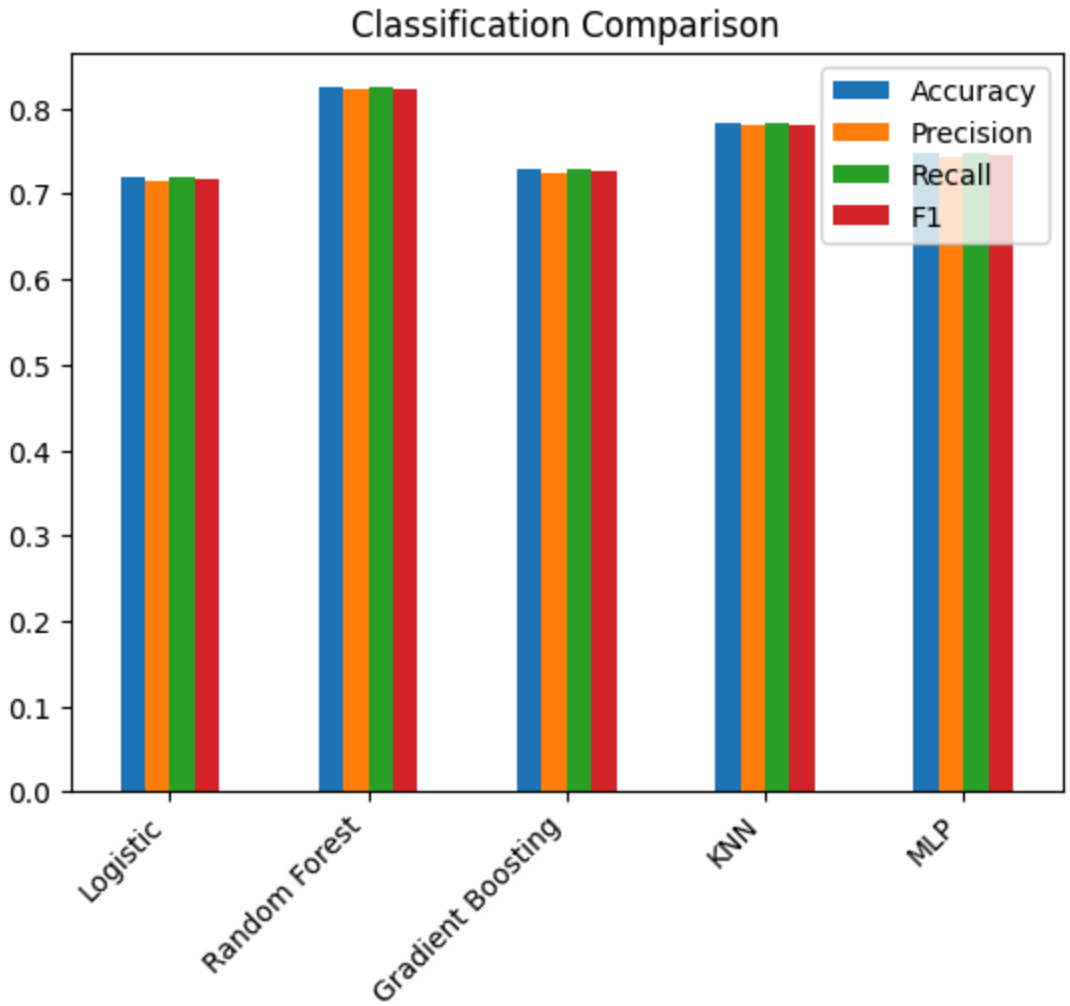
In [31]:
```
reg_df = pd.DataFrame(reg_results).T
clf_df = pd.DataFrame(clf_results).T

print("\nRegression Results:\n", reg_df)
print("\nClassification Results:\n", clf_df)
```

```
Regression Results:
                         R2        MAE       RMSE
Linear             0.649592  58.866812  78.800481
Ridge              0.649592  58.866819  78.800501
Random Forest      0.834770  34.748539  54.111127
Gradient Boosting  0.686209  54.939275  74.569666
XGBoost            0.790057  42.572649  60.994836

Classification Results:
                    Accuracy  Precision    Recall        F1
Logistic           0.720048   0.715531  0.720048  0.716849
Random Forest      0.823983   0.822132  0.823983  0.822522
Gradient Boosting  0.727959   0.725553  0.727959  0.726566
KNN                0.781930   0.780594  0.781930  0.781018
MLP                0.748621   0.744209  0.748621  0.745313
```

23. COMPARISON GRAPHS

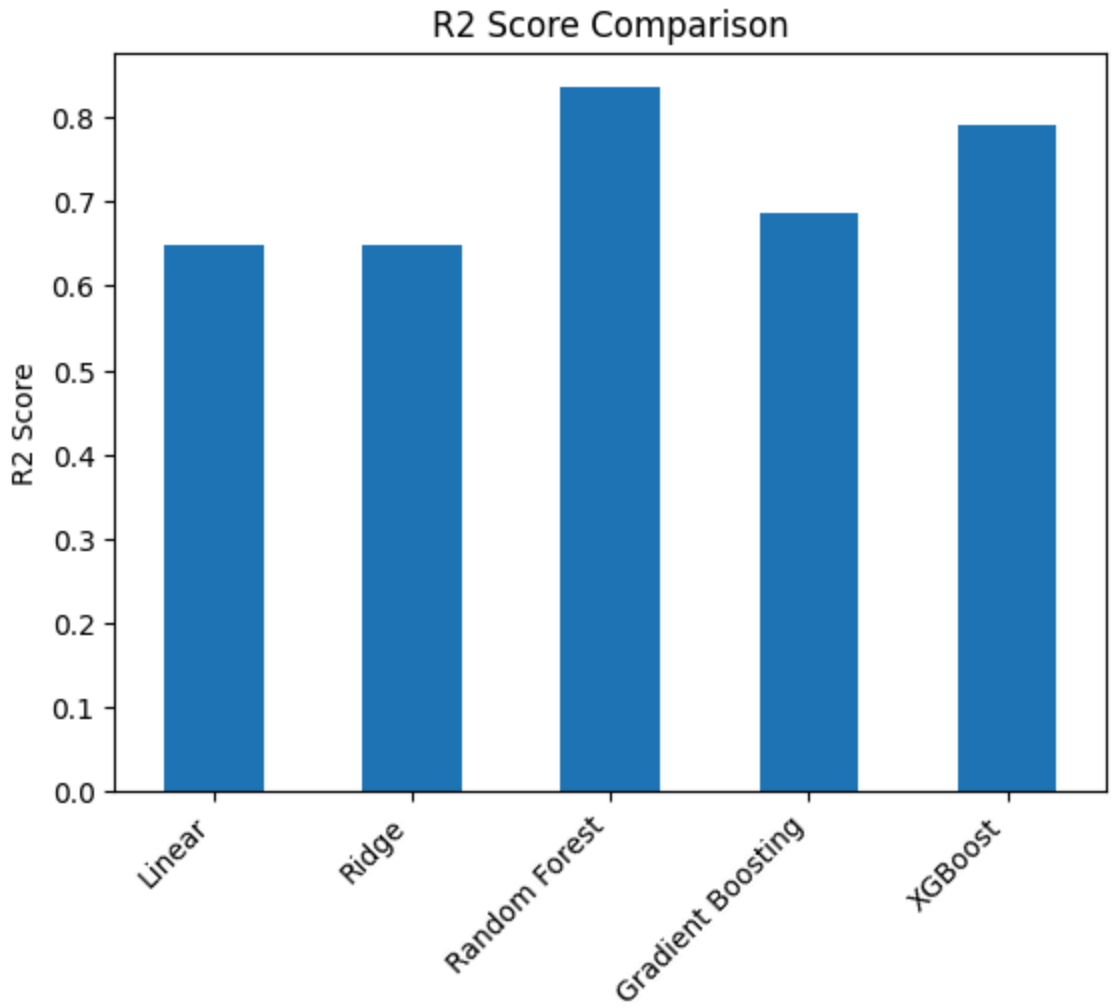
In [32]:
```
reg_df.plot(kind='bar', title="Regression Comparison")
plt.xticks(rotation=45, ha='right') # ha='right' se labels align ho jayenge

plt.show()

clf_df.plot(kind='bar', title="Classification Comparison")
plt.xticks(rotation=45, ha='right') # ha='right' se labels align ho jayenge
plt.show()
```

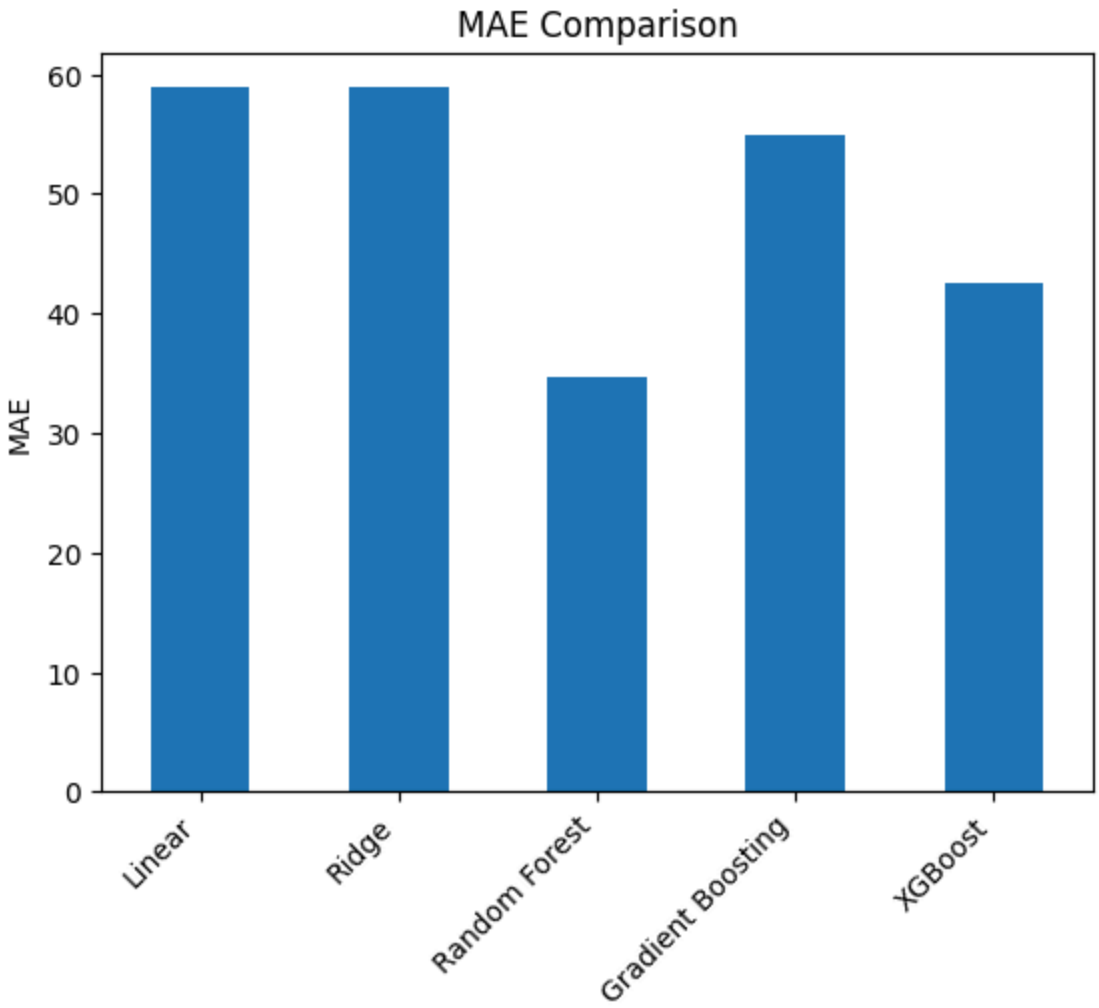

Regression Comparison
R2
MAE
RMSE
0
10
20
30
40
50
60
70
80
Linear
Ridge
Random Forest
Gradient Boosting
XGBoost

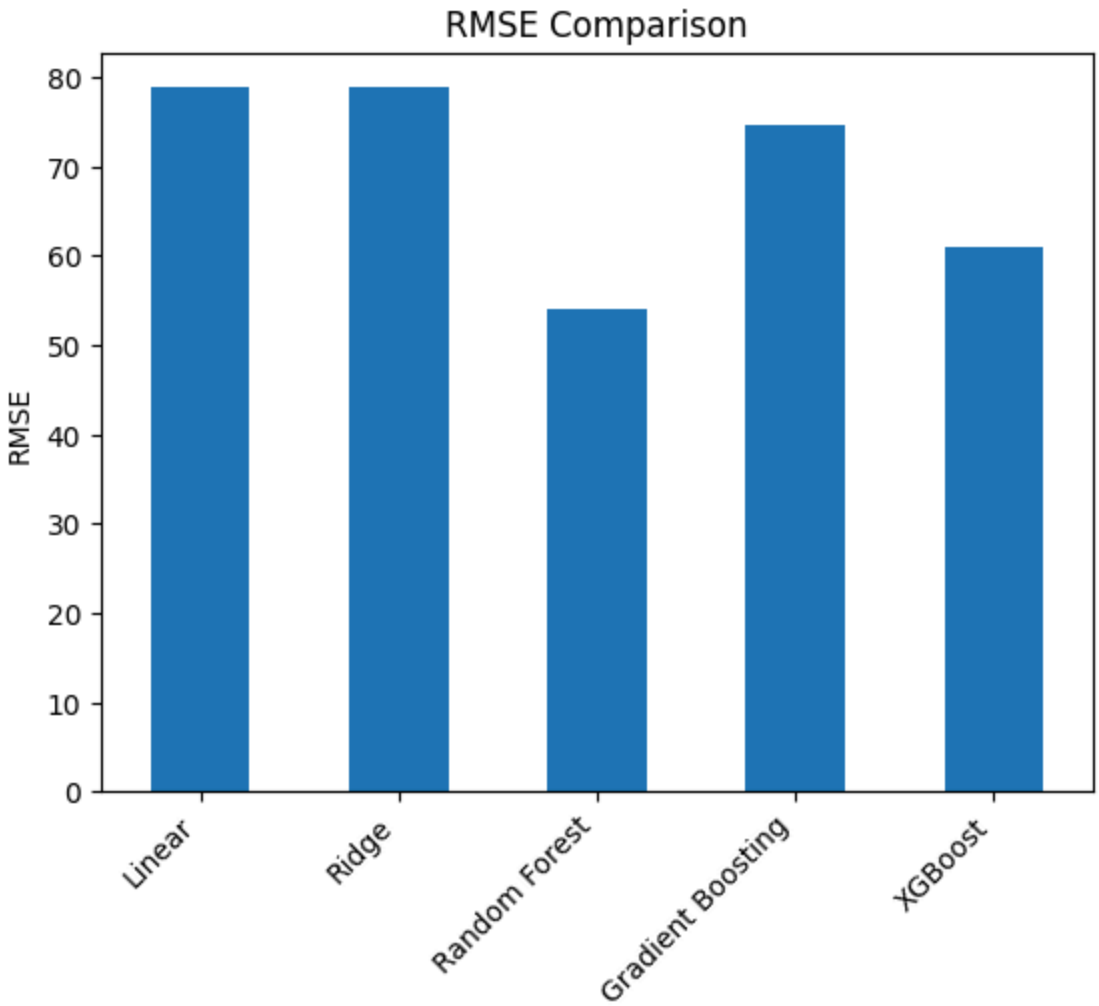


24.REGRESSION GRAPH CODE

In [33]:
```
# R2 Comparison
reg_df['R2'].plot(kind='bar', title="R2 Score Comparison")
plt.xticks(rotation=45, ha='right') # ha='right' se labels align ho jayenge
plt.ylabel("R2 Score")
plt.show()

# MAE Comparison
reg_df['MAE'].plot(kind='bar', title="MAE Comparison")
plt.xticks(rotation=45, ha='right') # ha='right' se labels align ho jayenge
plt.ylabel("MAE")
plt.show()

# RMSE Comparison
reg_df['RMSE'].plot(kind='bar', title="RMSE Comparison")
plt.xticks(rotation=45, ha='right') # ha='right' se labels align ho jayenge
plt.ylabel("RMSE")
plt.show()
```


R2 Score Comparison
R2 Score
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
Linear
Ridge
Random Forest
Gradient Boosting
XGBoost

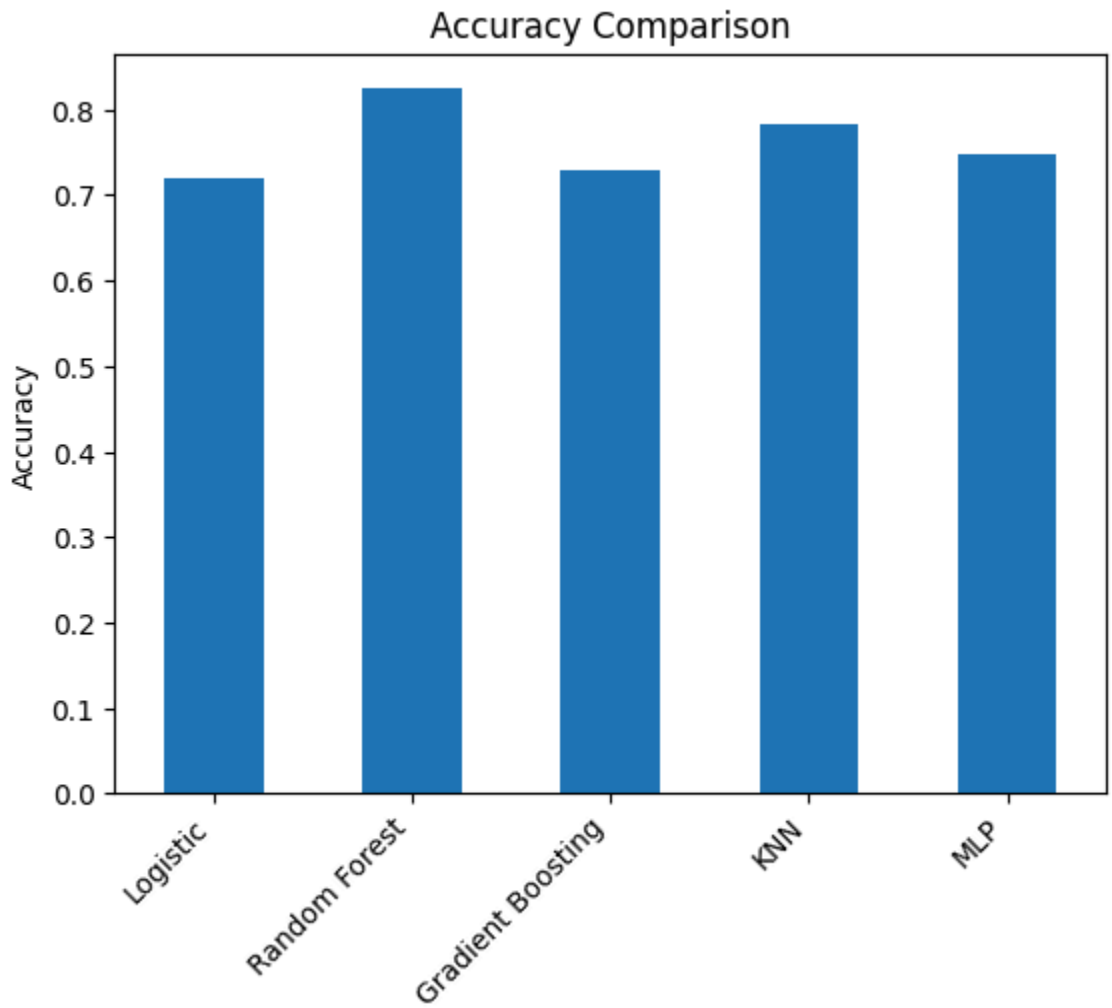

MAE Comparison
MAE
0
10
20
30
40
50
60
Linear
Ridge
Random Forest
Gradient Boosting
XGBoost

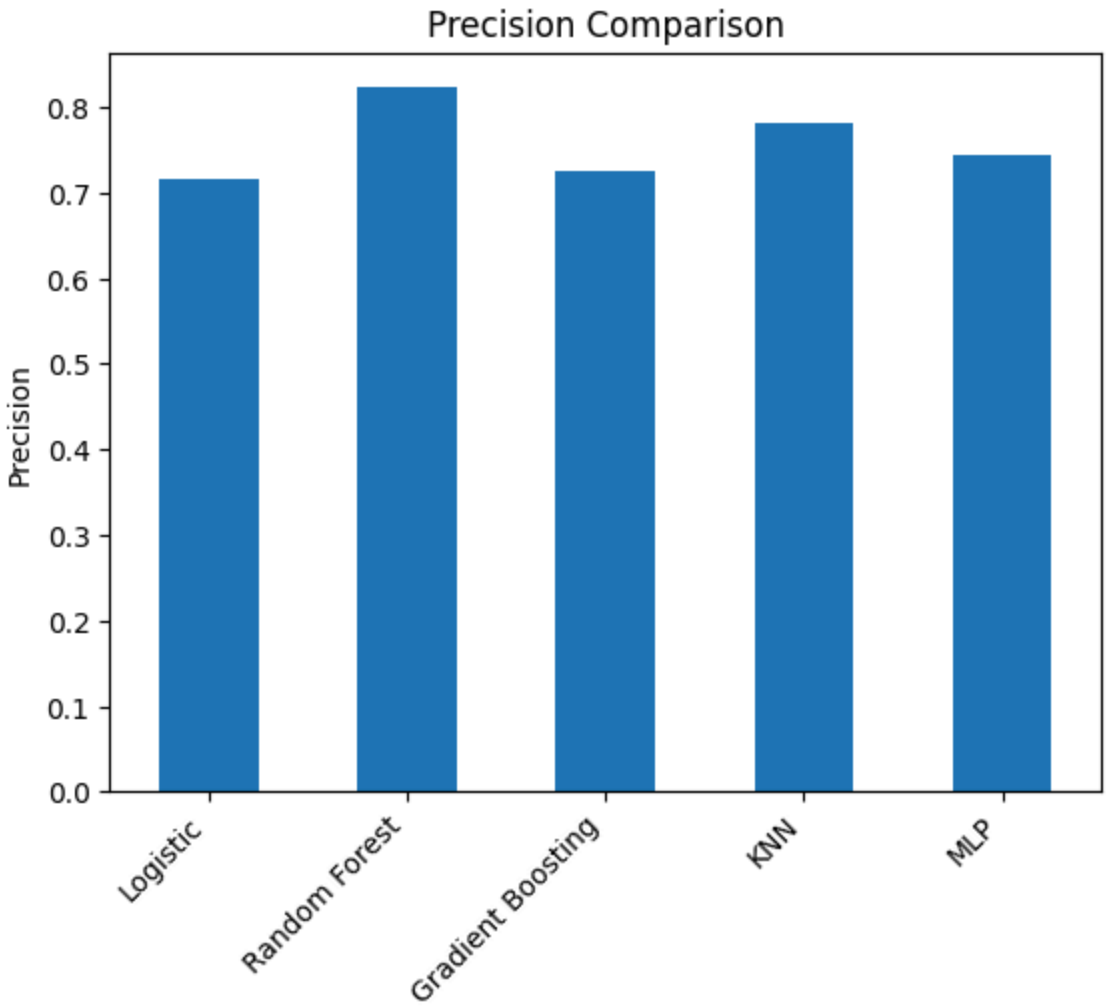


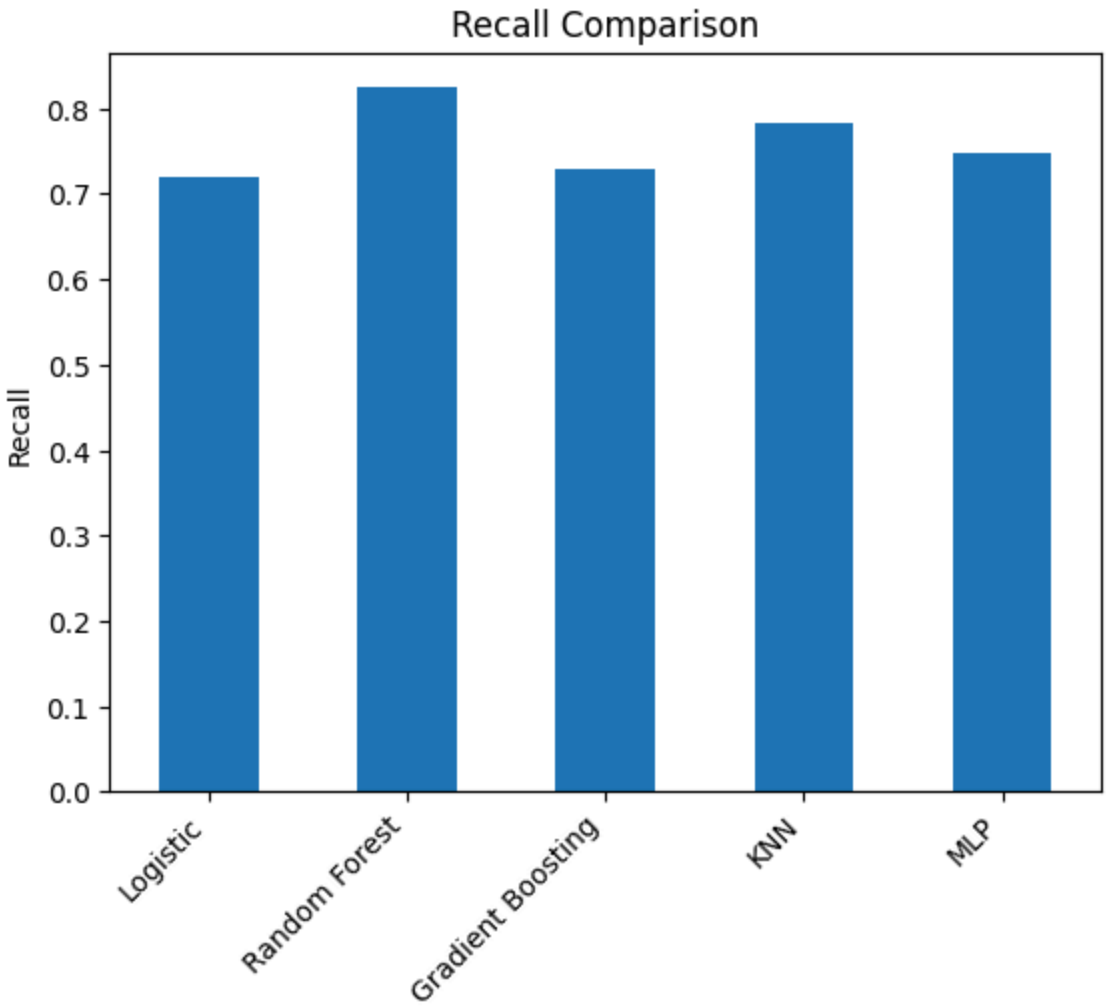
25. CLASSIFICATION GRAPH CODE

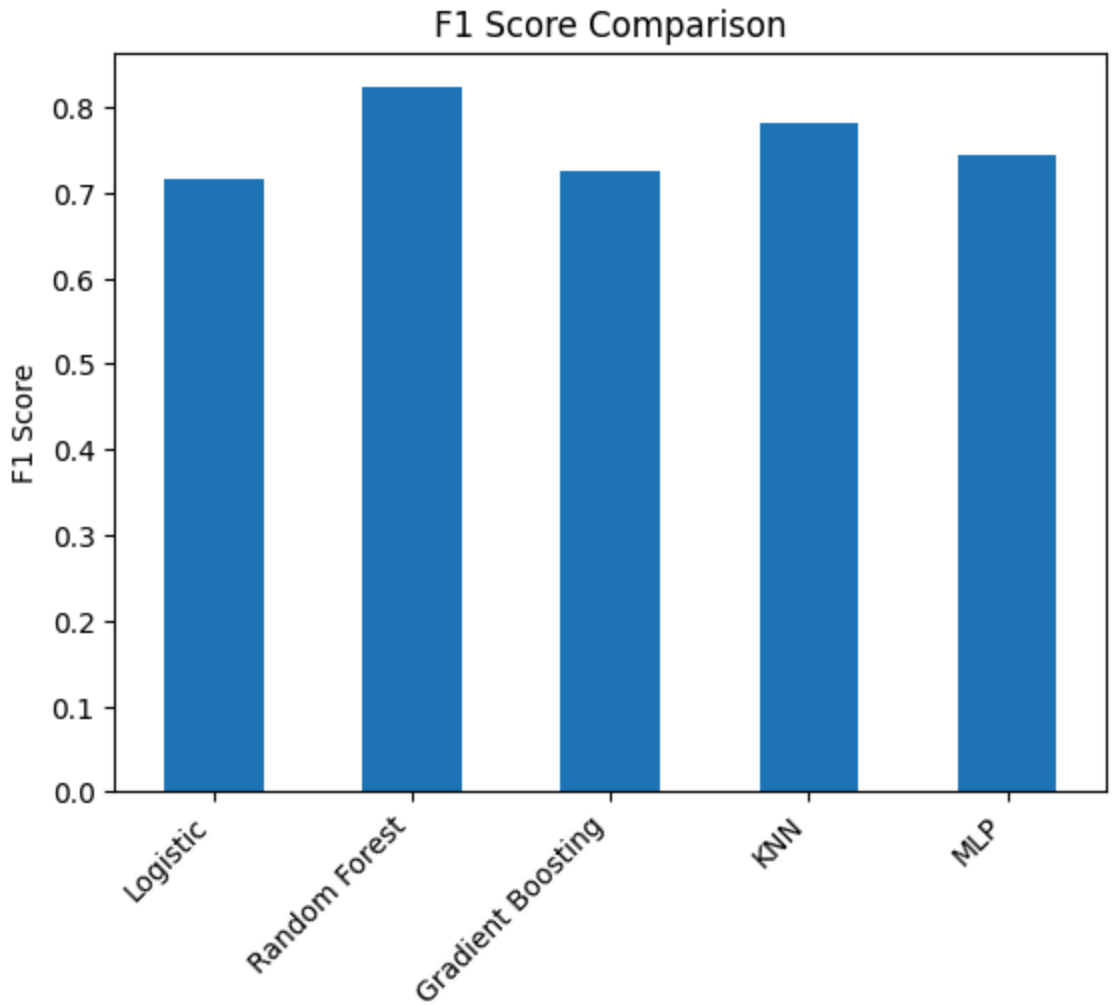
In [35]:
```
# Accuracy
clf_df['Accuracy'].plot(kind='bar', title="Accuracy Comparison")
plt.xticks(rotation=45, ha='right') # ha='right' se labels align ho jayenge
plt.ylabel("Accuracy")
plt.show()

# Precision
clf_df['Precision'].plot(kind='bar', title="Precision Comparison")
plt.xticks(rotation=45, ha='right') # ha='right' se labels align ho jayenge
plt.ylabel("Precision")
plt.show()

# Recall
clf_df['Recall'].plot(kind='bar', title="Recall Comparison")
plt.xticks(rotation=45, ha='right') # ha='right' se labels align ho jayenge
plt.ylabel("Recall")
plt.show()

# F1 Score
clf_df['F1'].plot(kind='bar', title="F1 Score Comparison")
plt.xticks(rotation=45, ha='right') # ha='right' se labels align ho jayenge
```

```
plt.ylabel("F1 Score")
plt.show()
```




Precision Comparison
Precision
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
Logistic
Random Forest
Gradient Boosting
KNN
MLP


Recall Comparison
Recall
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
Logistic
Random Forest
Gradient Boosting
KNN
MLP



26. HYPERPARAMETER TUNING

In [36]:
```
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import RandomizedSearchCV

# Parameter grid (reasonable ranges)
param_dist = {
    'n_estimators': [100, 200, 300],
    'max_depth': [5, 10, 15, None],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4],
    'max_features': ['sqrt', 'log2']
}

# Model
rf = RandomForestRegressor(random_state=42)

# Randomized Search
random_search = RandomizedSearchCV(
    estimator=rf,
    param_distributions=param_dist,
    n_iter=10,        # number of random combinations to try
    cv=3,             # reduced folds for speed
```

```
    verbose=1,
    random_state=42,
    n_jobs=-1         # use all CPU cores
)

# Fit
random_search.fit(X_train, y_train_r)

# Best results
print("Best Parameters:", random_search.best_params_)
print("Best CV Score:", random_search.best_score_)
```

```
Fitting 3 folds for each of 10 candidates, totalling 30 fits
Best Parameters: {'n_estimators': 100, 'min_samples_split': 5, 'min_samples_lea
f': 2, 'max_features': 'log2', 'max_depth': None}
Best CV Score: 0.7848245843273549
```

27 . Best model highlight

In [37]:
```
print("Best Regression Model (R2):")
print(reg_df['R2'].idxmax())

print("Best Classification Model (Accuracy):")
print(clf_df['Accuracy'].idxmax())
```

```
Best Regression Model (R2):
Random Forest
Best Classification Model (Accuracy):
Random Forest
```

28 .full ranking

In [38]:
```
# sort instead of just idxmax
print(reg_df.sort_values(by='R2', ascending=False))
print(clf_df.sort_values(by='Accuracy', ascending=False))
```

```
                         R2        MAE       RMSE
Random Forest      0.834770  34.748539  54.111127
XGBoost            0.790057  42.572649  60.994836
Gradient Boosting  0.686209  54.939275  74.569666
Linear             0.649592  58.866812  78.800481
Ridge              0.649592  58.866819  78.800501
                   Accuracy  Precision    Recall        F1
Random Forest      0.823983   0.822132  0.823983  0.822522
KNN                0.781930   0.780594  0.781930  0.781018
MLP                0.748621   0.744209  0.748621  0.745313
Gradient Boosting  0.727959   0.725553  0.727959  0.726566
Logistic           0.720048   0.715531  0.720048  0.716849
```